

Can AI Oversight Be Zero Knowledge?

Alessandro Chiesa
alessandro.chiesa@epfl.ch
EPFL

Ziyi Guan
ziyiguan@mit.edu
MIT

Burcu Yıldız
burcu.yildiz@epfl.ch
EPFL

October 2, 2026

Abstract

AI systems increasingly produce outputs from confidential data, such as a fitness-for-duty assessment from medical records or the predicted properties of a drug candidate from its secret structure. It is important to verify that such outputs are correct without revealing the underlying data. A recent line of work studies verification of AI outputs via interactive proofs and debate for oracle-aided computation, where correctness may depend on an oracle such as human judgment, a physical experiment, or the web. These works focus on verification by a verifier that runs much faster than the computation. However, such efficient verification is impossible for general oracle-aided computation, and these works therefore rely on additional assumptions. We focus instead on privacy: allowing the verifier to run in time polynomial in the computation, we ask whether interactive arguments for oracle-aided computation can be zero knowledge, so that the verifier learns nothing about the confidential data beyond the correctness of the output.

We prove that, in general, they cannot. In the random oracle model, there are no zero-knowledge proofs for all oracle-aided computations, even if both the prover and the verifier are allowed to run much longer than the computation itself. The impossibility extends to debate, a canonical model for scalable oversight.

On the positive side, we show that if the oracle attaches a cryptographic signature to each of its answers, then every oracle-aided computation can be verified in zero knowledge with an efficient prover and verifier, assuming only collision-resistant hash functions. Beyond privacy, this also gives an alternative approach to scalable oversight that relies neither on an honest opponent, as in debate, nor on the robustness of the computation, as in prior single-prover protocols.

Keywords: zero knowledge; relativized arguments; debate; oracle-aided computation; scalable oversight

Contents

1	Introduction	3
1.1	Our results	4
1.2	Related work	6
2	Technical overview	7
2.1	The lower bound	7
2.2	Debate	9
2.3	Construction with signed oracles	9
3	Preliminaries	11
3.1	Oracles, languages and relations	11
3.2	Turing machines and complexity classes with oracles	12
3.3	Relativized interactive arguments in an oracle model	13
3.4	Collision-resistant hash functions	15
3.5	Digital signatures	16
3.6	Probabilistically checkable proofs	16
3.7	Vector commitment schemes	17
4	Impossibility of witness hiding in the random oracle model	19
4.1	How to modify the oracle to insert a witness	20
4.2	Proof of Lemma 4.4	22
5	Zero-knowledge relativized arguments in the signed oracle model	26
5.1	A zero-knowledge argument of knowledge for the universal relation	27
5.2	Proof of Theorem 5.2	27
A	Impossibility of witness hiding for debate	30
	Acknowledgments	33
	AI use statement	33
	References	33

1 Introduction

AI systems now draft medical assessments [Sin+23; Moo+23; TTEGTT23; Pad+24; Vel+26], design candidate molecules [Zha+19; Sto+20; Won+24; Xu+25; JABJM24], and produce analyses that others rely on [NZ23; BLR25]. Although much of the data behind these outputs should remain confidential, it has repeatedly been exposed in practice. DeepSeek left a database with over a million log entries, including chat histories and API secrets, publicly accessible without authentication [Nag25]. Shared ChatGPT conversations and hundreds of thousands of shared Grok conversations became publicly searchable, exposing sensitive personal information [Sha25; MBW25]. Data brokers reportedly sold searchable chatbot transcripts harvested through browser extensions, including patient information entered by healthcare workers [Cla26]. Researchers have also shown how a crafted email or web page could trick a vulnerable AI assistant into leaking sensitive information [Rav25; Lak26].

These incidents motivate stronger privacy protections for data entrusted to AI systems. At the same time, there is growing demand for AI outputs that can be checked efficiently for correctness [AZWG21; ICA18; BCIP24; KCELMB24]. Even if the leaks described above are prevented, the verification process may itself expose sensitive data. Two notable examples illustrate this.

- *Occupational health.* Consider an AI system that drafts a fitness-for-duty assessment from an employee’s confidential medical records, which an occupational physician then reviews. The employer, or a licensing body, needs to know whether the employee is “fit to work at heights”, but the employee should not have to hand over their medical records. Yet reaching that conclusion requires a physician’s judgment about those very records.
- *Drug discovery.* Before committing to an AI-designed drug candidate, a prospective partner wants evidence that the molecule has the claimed properties. Checking them requires a laboratory *assay*: an experiment, such as a binding or toxicity test, that measures a property of the molecule. Molecular optimization treats the assay as a black-box oracle that the design algorithm queries at a cost [GFSC22]. But running the assay independently requires a physical sample of the molecule, and a sample can be analyzed to recover its structure, which is the trade secret.

Scalable oversight and oracle-aided computation. In both examples, checking the AI’s output requires an external judgment or measurement: the physician’s judgment, or the laboratory assay. The same dependence on external resources is the defining feature of *scalable oversight*, which asks how a human or a cheaper model can check an AI system’s output without carrying out the full task itself [CSA18; LKEMML18; IM25]. Following this line of work [BCIP24; CKX26], we model an external judgment or resource as a function f , called an *oracle*, which the computation can query in a black-box way.

In this oracle-aided setting, scalable oversight carries out its checks through protocols that build on *interactive proofs* [GMR89], in which a *prover* exchanges messages with an efficient *verifier* to establish a claim. The verifier accepts true claims from an honest prover with high probability, while any cheating prover has only a small chance of convincing it to accept a false claim. Here, the AI system acts as the prover: it claims that its output $\mathbb{x}$ is accepted by a public oracle-aided computation D , that is, $D^f(\mathbb{x}) = 1$. Both prover and verifier can query the same oracle. The goal is *succinct verification*: the verifier checks this claim reliably using much less time and fewer oracle queries than the computation D .

From succinctness to privacy. We ask whether the verifier can check such a claim while the underlying data remains confidential. The prover additionally knows confidential data (the *witness* $\mathbb{w}$) that the verifier does not receive, and the claim becomes “there exists $\mathbb{w}$ such that $D^f(\mathbb{x}, \mathbb{w}) = 1$ ”. In our examples, $\mathbb{w}$ is the medical records or the molecular structure, and $\mathbb{x}$ is the published conclusion, together with information

identifying the confidential object it concerns. The employer or business partner should be convinced that a valid witness supports the claim, without learning anything about the confidential data. This privacy requirement can be formalized using *zero knowledge* [GMR89]: the verifier can check the correctness of the claim without learning anything about the hidden data beyond what the claim itself reveals.

In the occupational-health example, the AI is the prover and the employer or licensing body is the verifier. The computation D takes the medical records w and the conclusion x , asks the physician, the oracle, the medical questions needed to check the assessment, and accepts if the conclusion is supported. In drug discovery, the AI is the prover and the prospective partner is the verifier. The computation D takes the molecule w and the claimed properties x , runs the laboratory assay, the oracle, on the molecule, and accepts if the measurements match the claim.

Our model: zero-knowledge oversight. Ideally, checking an AI’s output would be both succinct and zero knowledge. However, [BCG25] establish limitations on succinct verification for oracle-aided computations, even without a privacy requirement. But an employer or a business partner would be willing to spend more time checking a conclusion if that is what protecting the underlying data requires. We therefore relax the succinctness requirement and ask:

**Can every oracle-aided computation be verified in zero knowledge,
even if we do not require succinct verification?**

1.1 Our results

We answer this question in the negative: not every oracle-aided computation can be verified in zero knowledge, even if we do not require succinct verification. To work around this impossibility, we construct a protocol for succinct zero-knowledge verification under standard cryptographic assumptions by modifying the oracle to provide digital signatures on its answers. This requires the oracle, or a party operating it, to sign its answers, an assumption that we discuss below.

The impossibility result. Our impossibility result holds in the *random oracle model*, where the oracle is a uniformly random function. This is the standard model for relativized arguments [CT10; CCS22], and the model in which the limitation of [BCG25] on succinct verification is proved. We prove it even after weakening both the correctness and privacy requirements: (i) we only require the verifier to reject false claims from polynomial-time cheating provers, as in *relativized arguments*; (ii) we weaken privacy to *witness hiding*: the verifier may learn partial information about the confidential data, but should not be able to recover it in full or find alternative data that would pass the check for the same claim. Since zero knowledge implies witness hiding, this rules out zero-knowledge verification as well.

Theorem 1 (informal version of Theorem 4.1). *There exists an oracle-aided computation in the random oracle model that has no witness-hiding relativized argument, even when succinct verification is not required.*

Our lower bound concerns single-prover verification, the setting studied by [CKX26] for oracle-aided scalable oversight. Recall that [BCG25] rules out succinct single-prover verification for general oracle-aided computations. [CKX26] circumvents this limitation by assuming the computation is *robust*: its output tolerates a small fraction of incorrect oracle answers. This allows the verifier to check the computation using only a few oracle queries. But using a similar assumption in zero knowledge would in addition require the verifier’s queries and the answers it receives to reveal nothing about the hidden witness. This need not hold in general for computations we care about. For example, a query asking whether a molecule with a specified structure binds to a target reveals that entire structure to the verifier.

Another canonical model for scalable oversight is *debate*, where two provers compete to convince a verifier and correctness relies on the strong assumption that one prover acts honestly [ICA18; BCIP24]. This additional assumption allows debate to circumvent the limitation of [BCG25] on succinct verification. We can define privacy for debate in two ways, depending on which provers receive the secret witness. If both provers receive it, no privacy is possible, since either prover may be dishonest and send the witness to the verifier. If only the honest prover receives it, the weakest privacy requirement is that the opponent and the verifier cannot recover the witness even when both follow the protocol (a malicious opponent or verifier can in particular follow the protocol, and zero knowledge in particular hides the witness). Our impossibility rules out even this requirement, and hence every stronger one that an application may need. See Section 2.2 for details.

We therefore need a different approach to obtain zero knowledge for general oracle-aided computations.

Construction with signed oracles. [CT10] introduced the *signed random oracle model*, in which the random oracle returns each answer together with a digital signature on the query and its answer, and constructed relativized arguments with succinct verification in this model. We give a general oracle the same interface. A digital signature scheme lets the holder of a secret signing key sk produce a signature σ on any string m . Anyone holding the matching public verification key vk can then check that σ is a valid signature on m , and without sk nobody can produce a valid signature on a new string. An oracle f answers a query q with an answer a :

$$f(q) = a \text{ .}$$

Its signed version $\hat{f}$ returns the same answer together with a signature on the query and the answer:

$$\hat{f}(q) = (a, \sigma) \text{ where } \sigma \leftarrow \text{Sign}(sk, (q, a)) \text{ .}$$

The signature certifies that a is the oracle’s answer to q , and anyone holding vk can check this without querying f .

With this interface, we obtain zero-knowledge arguments with efficient proving and succinct verification for all oracle-aided computations.

Theorem 2 (informal version of Theorem 5.2). *Assume the existence of collision-resistant hash functions. For every oracle f and every oracle-aided computation, there exists a zero-knowledge single-prover argument with an efficient prover and succinct verification using the signed oracle $\hat{f}$.*

The signed oracle requires a party that holds the signing key and signs the oracle’s answers. When the oracle is a digital resource, such as a database or a model, this party can be trusted hardware that evaluates the oracle and signs its answers, as studied in cryptography [Kat07; PST17], in secure systems and databases [McK+13; CD16; BS14; PVC18], and in machine learning [MHKMPK21; Row26]. When the oracle is a person, as the physician in the occupational-health example, the person signs their judgments. When the oracle is a physical experiment, as the assay in the drug-discovery example, the laboratory that runs the experiment signs its results. The signer sees the queries, as the oracle does, and is trusted to sign only genuine answers and to keep its signing key private.

Beyond privacy, our construction provides a new alternative for scalable oversight. The limitation of [BCG25] on succinct verification has so far been circumvented by assuming an honest opponent, in debate [ICA18; BCIP24], or a robust computation [CKX26]. Signed oracles replace these assumptions by trust in a signer for the oracle’s answers. [CT10] showed that signatures suffice for succinct verification in the signed random oracle model; we show that they also suffice for zero knowledge, for every oracle relative to which collision-resistant hash functions exist.

1.2 Related work

Private verification of AI systems. Cryptographic verification of machine-learning computations includes zero-knowledge proofs of neural-network inference, from convolutional networks [LXZ21] to large language models [SLZ24], proof systems optimized for machine-learning operations [CWSK24], and zero-knowledge proofs of training [Gar+23; APKP24]. Other works establish that privacy and verifiability are compatible when the model is audited for certain criteria if the audit is specified as an ordinary computation: ZkAudit verifies computations over private models and training data [WSSHK24] and OATH certifies fairness of decisions while protecting model and user data [FSLH25]. On the other hand, our question concerns checks whose validity also depends on external oracle answers.

Scalable oversight. Scalable oversight concerns reliable supervision or evaluation of an AI model, even when the task/model exceeds a human judge’s capabilities. Works such as [CSA18; LKEMML18; Bur+24] focus on scalable supervision in training. On the other hand, [Bow+22] propose an empirical methodology for measuring progress on scalable oversight. [ICA18] propose debate as an alternative, using competing provers to help a weaker verifier. [BCIP24] formalize doubly-efficient debate for oracle-aided tasks, building on doubly-efficient interactive proofs [GKR15]. [BPHI25] examine the relevance of the debate model to alignment safety and identify the assumptions needed. [IM25] explain the role of relativization when verification depends on human judgments or other external resources. [Bar20] identifies the obfuscated-arguments problem for debate and suggests that “small” arguments are needed. Later work [BCIPCLX25] addresses obfuscation by prover-estimator debate. [CKX26] show that we can consider the simpler model with a single prover if the oracle-aided computation is robust or the oracle is low-degree. These works principally address correctness and the cost of oversight, rather than simulation-based privacy for a confidential witness. We keep their oracle-aided formulation but ask whether the witness can remain hidden even when the verifier is allowed a polynomial overhead.

Relativized arguments. The study of relativized arguments is also motivated by proofs that verify other proofs, as in incrementally verifiable computation and proof-carrying data [Val08; CT10; BCCT13]. The issue is that the computation being proved may invoke the same oracle as the proof system. Positive constructions exploit signed responses [CT10; MMZ25], low-degree encodings [CCS22], or arithmetized oracle access [CCGOS23]. [CGSY24] analyze the security of recursive proof composition in oracle models. Oracle-relative formulations also arise in the study of white-box security for the Fiat–Shamir transformation [AY25].

On the negative side, [CL20] establish limitations of probabilistically checkable proofs and interactive oracle proofs in relativized worlds. Most directly, [BCG25] rule out succinct relativized arguments in the random oracle model [BR93], even with interaction and computational soundness. Our lower bound is complementary: it imposes no succinctness requirement and rules out even witness hiding [FS90], a requirement weaker than zero knowledge [GMR89] (Lemma 3.19).

2 Technical overview

We explain the main ideas behind our results. Section 2.1 explains the proof of the lower bound (Theorem 1), Section 2.2 explains why the lower bound extends to debate, and Section 2.3 explains the construction with signed oracles (Theorem 2).

2.1 The lower bound

Recall that our goal is to construct a *relativized interactive argument* for an oracle-aided computation D with access to an arbitrary oracle f . The prover receives $(\mathbb{x}, \mathbb{w})$, while the verifier receives only $\mathbb{x}$; both can query f . The prover must convince the verifier that there exists a private witness $\mathbb{w}$ such that $D^f(\mathbb{x}, \mathbb{w}) = 1$. Correctness requires the verifier to accept true claims from an honest prover (*completeness*) and reject false claims from computationally bounded cheating provers (*soundness*), with high probability. Zero knowledge requires the interaction to reveal nothing about $\mathbb{w}$. Our lower bound rules out even *witness hiding*, the weaker requirement of Section 1.1: the verifier may learn partial information about a witness, as long as what it learns is not enough to recover an entire valid witness. Since zero knowledge implies witness hiding (Lemma 3.19), ruling out witness hiding rules out zero knowledge as well, and we speak of witness hiding from here on.

Our impossibility result comes from a conflict between correctness and witness hiding. We choose f to be a *random oracle*: a function sampled uniformly at random and then kept fixed. A protocol for arbitrary oracles would have to work for this family too. We construct a computation $C_\star$ for which any correct protocol forces the verifier to learn a valid witness with constant probability. A witness cannot be found from the public input and oracle access alone, so a protocol that reveals one to the verifier gives it something it could never have computed itself. This is exactly what witness hiding forbids.

A hard computation. We use the computation underlying the relation of [BCG25]. Fix the public input $\mathbb{x} = 0^n$ and let candidate witnesses be m -bit strings. The computation accepts exactly when

$$C_\star^f(0^n, \mathbb{w}) = 1 \iff f(\mathbb{w} \| i)_1 = 0 \text{ for every } i \in \{1, \dots, m\}. \quad (1)$$

Here $\|$ denotes concatenation, i is written as a binary string of fixed length, and the subscript 1 denotes the first bit of the oracle's answer.

Think of these bits as a table with 2^m rows and m columns (Figure 1). Each row is labeled by a candidate witness $\mathbb{w}$, and column i contains $f(\mathbb{w} \| i)_1$. The claim is that the table contains an all-zero row, and the witness is the label of such a row. Given the witness, $C_\star$ checks the claim by reading the m bits in that row. Without the witness, finding an all-zero row is hard: the rows are independent, each is all zero with

	$i = 1$	2	$\dots$	m	
$0 \dots 00$	1	0	$\dots$	1	
$0 \dots 01$	0	1	$\dots$	0	
	$\vdots$	$\vdots$		$\vdots$	
$\mathbb{w}$	0	0	$\dots$	0	← all-zero row: its label $\mathbb{w}$ is a witness
	$\vdots$	$\vdots$		$\vdots$	
$1 \dots 11$	1	1	$\dots$	0	

Figure 1: The table of bits read by the hard computation $C_\star$: 2^m rows, one per candidate witness, and m columns. A witness is the label of an all-zero row (shaded); the boxed entry is what one query reads.

probability 2^{-m} , and polynomially many oracle queries have only an exponentially small chance of finding one.

The table is a simplified version of the drug-discovery example. A row label is the structure of a molecule, a column is a test, and a zero means that the molecule passed that test. The claim is that some molecule passes every test, and a witness is such a molecule. Asking for a test result means submitting the structure, so a verifier cannot ask about a molecule without revealing it. Our lower bound also assumes that the test outcomes are independent random bits, which is what the random oracle provides.

The verifier must read the witness row. Suppose, towards a contradiction, that $(\mathcal{P}, \mathcal{V})$ is a witness-hiding relativized argument for $C_\star$. Consider a cheating prover $\tilde{\mathcal{P}}$ that chooses a random row label w and runs $\mathcal{P}$ on $(0^n, w)$, but answers its oracle queries using a modified oracle f' (Figure 2). This oracle agrees with f except that the first bits of all answers in row w are set to zero. $\tilde{\mathcal{P}}$ can supply these altered answers to its internal copy of $\mathcal{P}$; the verifier $\mathcal{V}$ still queries the real oracle f .

Now suppose the real table has no all-zero row, which happens with constant probability. Then $C_\star^f(0^n, w') = 0$ for every w' , while $C_\star^{f'}(0^n, w) = 1$. Consider two interactions in which the prover sends exactly the same messages.

- *The honest interaction* between $\mathcal{P}$ on input $(0^n, w)$ and $\mathcal{V}$ on input 0^n , both with oracle f' . By completeness, its decision is 1 with high probability, since $C_\star^{f'}(0^n, w) = 1$.
- *The cheating interaction* between $\tilde{\mathcal{P}}$ on input 0^n and $\mathcal{V}$ on input 0^n , both with oracle f ($\tilde{\mathcal{P}}$ feeds f' to its internal copy of $\mathcal{P}$). By soundness against $\tilde{\mathcal{P}}$, its decision is 0 with high probability, since $C_\star^f(0^n, w') = 0$ for every w' .

The two oracles agree outside row w . As long as $\mathcal{V}$ does not query row w , it receives the same oracle answers and the same prover messages in both interactions, and therefore reaches the same decision. Since the two decisions must differ, $\mathcal{V}$ queries row w with high probability in the honest interaction with f' . Making that query requires $\mathcal{V}$ to know w . So the honest verifier learns a witness that it could not have found on its own, which violates witness hiding, and hence zero knowledge as well (Lemma 3.19). Nothing in the argument required $\mathcal{V}$ to be succinct, which is why the theorem needs no succinctness requirement (Theorem 4.1).

	real oracle f queried by the verifier					modified oracle f' fed to the internal honest prover			
$0 \dots 00$	1	0	$\dots$	1		1	0	$\dots$	1
$0 \dots 01$	0	1	$\dots$	0		0	1	$\dots$	0
$\vdots$	$\vdots$	$\vdots$		$\vdots$		$\vdots$	$\vdots$		$\vdots$
w	1	0	$\dots$	1	differ only here	0	0	$\dots$	0
$\vdots$	$\vdots$	$\vdots$		$\vdots$		$\vdots$	$\vdots$		$\vdots$
$1 \dots 11$	1	1	$\dots$	0		1	1	$\dots$	0

no all-zero row: the claim is false row w is all zero: the claim is true

Figure 2: The two worlds in the proof. Left: the real oracle f , which the verifier queries; no row is all zero, so the claim is false. Right: the oracle f' that the cheating prover feeds to its internal copy of the honest prover; row w is overwritten with zeros, so the claim is true and w is a witness. The tables agree outside row w , so a verifier that never reads that row receives the same oracle answers, and the same prover messages, in both worlds.

2.2 Debate

We now show that the lower bound extends to debate. This contrasts with succinctness: a second prover lets debate achieve succinct verification where a single prover cannot [BCIP24], but it does not make privacy possible. Recall that a *debate* involves a prover $\mathcal{P}_1$, an opponent $\mathcal{P}_2$, and a verifier $\mathcal{J}$ [ICA18; BCIP24]. All three know the public input $\mathbb{x}$ and can query the oracle f . The prover $\mathcal{P}_1$ tries to establish that there exists a witness $\mathbb{w}$ such that $D^f(\mathbb{x}, \mathbb{w}) = 1$, while the opponent $\mathcal{P}_2$ challenges this claim. Completeness requires that, if the claim is true, the honest prover $\mathcal{P}_1$ convinces $\mathcal{J}$ to accept regardless of the opponent's strategy. Soundness requires that, if the claim is false, the honest opponent $\mathcal{P}_2$ makes $\mathcal{J}$ reject regardless of the cheating prover's strategy.

There are two natural ways to define privacy for debate, depending on which provers receive the secret witness. One option is to give $\mathbb{w}$ to both $\mathcal{P}_1$ and $\mathcal{P}_2$ and require that it remain hidden from $\mathcal{J}$. This fails in general because either prover may be dishonest, and a dishonest prover can simply send $\mathbb{w}$ to $\mathcal{J}$. The other option is to give $\mathbb{w}$ only to $\mathcal{P}_1$, while $\mathcal{P}_2$ and $\mathcal{J}$ receive only $\mathbb{x}$ and oracle access. In this case, privacy must hold against $\mathcal{P}_2$ and $\mathcal{J}$ together, since they may cooperate. We require only that $\mathcal{P}_2$ and $\mathcal{J}$, when both follow the protocol, cannot recover $\mathbb{w}$ from what they see together. This is the weakest privacy requirement for this option: a malicious $\mathcal{P}_2$ or $\mathcal{J}$ can in particular follow the protocol, so privacy against malicious parties implies it, and zero knowledge, which requires their view to reveal nothing about $\mathbb{w}$, in particular requires that it not reveal $\mathbb{w}$. Ruling it out therefore rules out every stronger requirement.

For this second definition, our lower bound applies by a reduction to the single-prover case. Let $\mathcal{V}$ be the verifier that runs $\mathcal{P}_2$ and $\mathcal{J}$ internally, exchanges messages with $\mathcal{P}_1$ on their behalf, and outputs the decision of $\mathcal{J}$. Then $(\mathcal{P}_1, \mathcal{V})$ is a single-prover argument for the same claim:

- it is complete, because the honest $\mathcal{P}_1$ convinces $\mathcal{J}$ against every opponent, in particular against $\mathcal{P}_2$;
- it is sound, because the soundness of the debate is exactly the guarantee that $\mathcal{P}_2$ defeats every cheating $\tilde{\mathcal{P}}_1$;
- it is witness hiding if the debate is, because $\mathcal{V}$ knows exactly what $\mathcal{P}_2$ and $\mathcal{J}$ know together.

Hence a witness-hiding debate for the hard computation $C_\star$ would give a witness-hiding argument for it, which Theorem 1 rules out. Corollary A.8 states the result, and Section A gives the reduction (Lemma A.7).

2.3 Construction with signed oracles

We now show how a signed oracle makes succinct zero-knowledge verification possible for every oracle-aided computation (Theorem 2). Recall from Section 2.1 that the lower bound rests on one fact: the only way for the verifier to check that an oracle answer is genuine is to make the query, and for some computations the query requires knowing the witness. A signed oracle $\hat{f}$ (Section 1.1) removes this obstacle. Every answer comes with a signature on the query and the answer, and anyone holding vk can check the signature without querying the oracle, so certifying an answer no longer requires querying the oracle itself.

Here is a first attempt. The prover runs D on $(\mathbb{x}, \mathbb{w})$ with the signed oracle and sends the verifier the entire record of the execution: every query it made, the answer, and the signature. The verifier checks that the record is an accepting execution of D and that every signature in it is valid under vk , without querying the oracle. This is sound, but it is not zero knowledge in general: the record contains the prover's queries, which may contain the confidential data (in the computation of Section 2.1, the queries are the witness itself). So the prover must keep $\mathbb{w}$ and the record to itself, and instead prove to the verifier that it holds a witness and a record such that (i) the record is an accepting execution of D on $(\mathbb{x}, \mathbb{w})$ and (ii) every signature in the record is valid under vk . Given $\mathbb{w}$ and the record, checking (i) and (ii) is an ordinary computation that never calls f .

Specifically, the prover proves that for $(\mathbb{x}, \text{vk})$ there exist $\mathbb{w}$ and $((q_i, a_i, \sigma_i))_i$ such that D on $(\mathbb{x}, \mathbb{w})$ makes the queries $q_1, q_2, \dots$, receives the answers $a_1, a_2, \dots$, and accepts, and $\text{Verify}(\text{vk}, (q_i, a_i), \sigma_i) = 1$ for every i . This is an NP statement that does not depend on any oracle: given $\mathbb{w}$ and $((q_i, a_i, \sigma_i))_i$, it can be checked in polynomial time. Thus we can use standard techniques from cryptography to prove it, namely a *zero-knowledge argument of knowledge*. Such an argument lets a prover convince a verifier that it knows a valid witness for an NP statement while revealing nothing about that witness beyond the truth of the statement. Arguments of this kind with succinct verification exist assuming collision-resistant hash functions [Kil92; BG08].

We apply this argument to the statement above, with public input $(\mathbb{x}, \text{vk})$ and private witness $(\mathbb{w}, ((q_i, a_i, \sigma_i))_i)$. The resulting protocol has the three properties we want:

- it is sound, because the knowledge requirement lets us extract the record from a convincing prover, and for a false claim the record must contain a fabricated oracle answer with a valid signature, a forgery that the security of the signature scheme rules out;
- it is zero knowledge, because the argument hides the original witness, the queries, and the answers;
- it has succinct verification, because the verifier never queries the underlying oracle f , and checks a succinct argument.

Combining a signed oracle with an ordinary zero-knowledge argument of knowledge therefore gives succinct zero-knowledge verification of the original oracle-aided computation (Theorem 5.2). Our construction keeps the argument of knowledge simple to demonstrate feasibility; replacing it by a modern hash-based succinct argument makes the protocol concretely efficient (Remark 5.4).

3 Preliminaries

Some of our definitions in this section are taken verbatim from [BCG25].

For a set S , we denote by 2^S the set of all subsets of S . All sets considered in this paper are finite. For every finite set, we can define a bijective binary encoding. Therefore, without loss of generality, we assume every set considered in this paper is a subset of $\{0, 1\}^*$.

We write $x \leftarrow \mathcal{D}$ to denote that x is a sample of the distribution $\mathcal{D}$. We write $x \leftarrow \mathcal{X}$ to denote that x is sampled uniformly at random from the non-empty set $\mathcal{X}$. We write $x \in \mathcal{D}$ to denote that x is in the support of $\mathcal{D}$, i.e. the value x is sampled with non-zero probability from $\mathcal{D}$.

3.1 Oracles, languages and relations

Definition 3.1. For all finite sets X and Y , the set $\text{Func}(X, Y)$ is the set of all functions $f: X \rightarrow Y$.

Definition 3.2 (Oracle). An **oracle** is a collection $\mathcal{O} = \{\mathcal{O}_{\lambda,n}\}_{\lambda \in \mathbb{N}, n \in \mathbb{N}}$ where each $\mathcal{O}_{\lambda,n}$ is a distribution over functions $\text{Func}(X_{\lambda,n}, Y_{\lambda,n})$ for some finite sets $X_{\lambda,n}, Y_{\lambda,n}$.

We write $f \in \mathcal{O}$ to denote that f is in the support of $\mathcal{O}_{\lambda,n}$ for some $\lambda \in \mathbb{N}, n \in \mathbb{N}$. An oracle $\mathcal{O}$ has **input size bound** ℓ_{in} if for every $\lambda, n \in \mathbb{N}$, $X_{\lambda,n} \subseteq \{0, 1\}^{\leq \ell_{\text{in}}(\lambda,n)}$.

The empty oracle $\mathcal{O}_\perp$ is the oracle whose only function $f \in \mathcal{O}_\perp$ is the function with empty domain.

Definition 3.3 (Relativized language). Let $\mathcal{O}$ be an oracle. A relativized language $L_\mathcal{O}$ is a collection of languages indexed by functions $f \in \mathcal{O}_{\lambda,n}$ for $\lambda, n \in \mathbb{N}$, namely,

$$L_\mathcal{O} := \{L_f\}_{\lambda,n \in \mathbb{N}, f \in \mathcal{O}_{\lambda,n}}.$$

Definition 3.4 (Relativized relation). Let $\mathcal{O}$ be an oracle. A relativized relation $R_\mathcal{O}$ is a collection of relations indexed by a security bound $\lambda \in \mathbb{N}$ and instance size $n \in \mathbb{N}$ and functions $f \in \mathcal{O}_{\lambda,n}$, namely,

$$R_\mathcal{O} := \{R_f\}_{\lambda,n \in \mathbb{N}, f \in \mathcal{O}_{\lambda,n}}.$$

The relativized language corresponding to the relativized relation $R_\mathcal{O}$, denoted $L(R_\mathcal{O})$, is defined as follows:

$$L(R_\mathcal{O}) := \{L(R_f)\}_{\lambda,n \in \mathbb{N}, f \in \mathcal{O}_{\lambda,n}},$$

where each $L(R_f) := \{\mathbb{x} : \exists \mathbb{w} \text{ s.t. } (\mathbb{x}, \mathbb{w}) \in R_f\}$.

Remark 3.5. Let R be a relation and $\mathcal{O}$ be an oracle. We can view R as a relativized relation $\{R_f\}_{f \in \mathcal{O}}$ where $R_f := R$ for every $f \in \mathcal{O}$.

Definition 3.6. Let $R_\mathcal{O}$ be a relativized relation. For $f \in \mathcal{O}$ and $\mathbb{x} \in \{0, 1\}^*$, the **set of witnesses for $\mathbb{x}$ in R_f** is $\text{Witness}^{R_\mathcal{O}}(f, \mathbb{x}) := \{\mathbb{w} \in \{0, 1\}^* : (\mathbb{x}, \mathbb{w}) \in R_f\}$.

Definition 3.7. For every family of pairs of sets $((X_{\lambda,n}, Y_{\lambda,n}))_{\lambda,n \in \mathbb{N}}$, the **random oracle over $((X_{\lambda,n}, Y_{\lambda,n}))_{\lambda,n \in \mathbb{N}}$** is the collection $\{\mathcal{U}_{\lambda,n}\}_{\lambda,n \in \mathbb{N}}$, where each $\mathcal{U}_{\lambda,n}$ is the uniform distribution over $\text{Func}(X_{\lambda,n}, Y_{\lambda,n})$.

3.2 Turing machines and complexity classes with oracles

To define the complexity class $\text{NTIME}^{\mathcal{O}}[t]$, we consider Turing machines that can query oracles. An oracle machine is a machine that has *black-box* access to a function $f : X \rightarrow Y$ that the machine can query. Each query costs the machine a single computational step.

Definition 3.8. Let $\mathcal{O}$ be an oracle. An **oracle Turing machine** M is a Turing machine that has two additional special tapes, the oracle query tape and the oracle answer tape, and two additional special states, the QUERY state and the ANSWER state. Given input $\mathbb{x} \in \{0, 1\}^*$ and oracle access to a function $f \in \mathcal{O}_{\lambda, n}$ for some λ, n , $M^f(\mathbb{x})$ works as follows:

- The input $\mathbb{x}$ is written in the input tape.
- The execution of M proceeds as a normal Turing machine computation except that, when M intends to make a query to the oracle, then it enters the QUERY state.
- In the QUERY state, M writes its query $q \in \{0, 1\}^*$ on the query tape.
- In the following step, M enters the ANSWER state and the content in the answer tape becomes $f(q)$ if q is in the domain of f and empty otherwise.

We call the function that maps the tuple of the input x and the oracle answers to the first $i - 1$ oracle queries to M 's i -th query the i -th query function of M .

We extend the definition of oracle Turing machines to deterministic oracle Turing machines, nondeterministic oracle Turing machines, and probabilistic oracle Turing machines with the standard definitions of deterministic Turing machines, nondeterministic Turing machines, and probabilistic Turing machines in prior literature. Throughout this paper, we focus on oracle algorithms, which are implemented as oracle Turing machines.

Definition 3.9. The **view** of a probabilistic interactive oracle algorithm A is a function View_A that maps the oracle f , the input z , the messages $(a_i)_{i \in [k]}$ that the algorithm receives, and the randomness r to the tuple $(z, r, (a_i)_{i \in [k]}, \mu := ((q_i, f(q_i)))_{i \in [q]})$, where $(q_i)_{i \in [q]}$ is the list of all oracle queries of $A^f(z)$, $A^f(z, a_1), \dots$, and $A^f(z, a_1, \dots, a_k)$.

For every oracle f , input z , messages $(a_i)_{i \in [k]}$ and randomness r , the output of $A^f(z, (a_i)_{i \in [k]}; r)$ is a function of $\text{View}_A(f, z, (a_i)_{i \in [k]}, r)$.

Definition 3.10. Let A_0 and A_1 be interactive oracle algorithms, interacting for k rounds, with respective input domains Z_0 and Z_1 and randomness complexities ar_0 and ar_1 . For $f_0, f_1 \in \mathcal{O}$, $z_0 \in Z_0$, $z_1 \in Z_1$, we denote the random experiment

$$\left\{ \begin{array}{l} r_0 \leftarrow \{0, 1\}^{\text{ar}_0} \\ r_1 \leftarrow \{0, 1\}^{\text{ar}_1} \\ \text{For } i \in [k] : \\ \quad b_i := A_0^{f_0}(z_0, (a_j)_{j < i}; r_0) \\ \quad a_i := A_1^{f_1}(z_1, (b_j)_{j \leq i}; r_1) \\ d := a_k \\ \text{tr} := (b_1, a_1, \dots, b_k, a_k) \end{array} \right\}$$

by $d \xleftarrow{r_0, r_1, \text{tr}} \langle A_0^{f_0}(z_0), A_1^{f_1}(z_1) \rangle$. When r_0, r_1, tr are irrelevant, we omit them from the notation.

If $\text{ar}_0 = \text{ar}_1 = 0$, we write $d \xleftarrow{\text{tr}} \langle A_0^{f_0}(z_0), A_1^{f_1}(z_1) \rangle$.

Nondeterministic time. A nondeterministic machine M is an **ND-decider** for a relation R if for every $(\mathbb{x}, \mathbb{w})$ it holds that $M(\mathbb{x}, \mathbb{w}) = 1$ if and only if $(\mathbb{x}, \mathbb{w}) \in R$, where $M(\mathbb{x}, \mathbb{w})$ denotes the decision of M on the input $\mathbb{x}$ on the branch $\mathbb{w}$.

Definition 3.11 (Relativized NTIME). *Let $\mathcal{O}$ be an oracle. Let t be a time-constructible function. The complexity class $\text{NTIME}^{\mathcal{O}}[t]$ is the set of relativized relations $R_{\mathcal{O}} = \{R_f\}_{\lambda, n \in \mathbb{N}, f \in \mathcal{O}_{\lambda, n}}$ such that there exists an oracle nondeterministic machine M that runs in time $O(t(n))$ and, for every $\lambda, n \in \mathbb{N}$,*

$$\Pr \left[M^f \text{ is an ND-decider for } R_f \mid f \leftarrow \mathcal{O}_{\lambda, n} \right] = 1 .$$

We call M an **ND-decider** for $R_{\mathcal{O}}$.

Definition 3.12 (Universal relation [BG08]). *The **universal relation** $R_{\mathcal{U}}$ is the set of pairs $((M, \mathbb{x}, t), \mathbb{w})$ such that the Turing machine M accepts $(\mathbb{x}, \mathbb{w})$ within t steps and $|\mathbb{w}| \leq t$.*

3.3 Relativized interactive arguments in an oracle model

Let $\mathcal{O}$ be an oracle. A **relativized interactive argument** in the oracle model $\mathcal{O}$ for a relativized relation $R_{\mathcal{O}}$ is a pair of interactive oracle algorithms $\text{ARG} = (\mathcal{P}, \mathcal{V})$ that satisfies the following properties. Both algorithms have access to the same oracle function $f \in \mathcal{O}_{\lambda, n}$. They interact as in Definition 3.10. Our arguments have no trusted setup: public parameters, such as those of a commitment scheme, are sampled by the verifier and sent as its first message, and the prover aborts if they are not well formed, so zero knowledge has to hold for every well-formed choice of the parameters.

Definition 3.13 (Completeness). *$\text{ARG} = (\mathcal{P}, \mathcal{V})$ for a relativized relation $R_{\mathcal{O}}$ has **completeness error** α if for every security parameter $\lambda \in \mathbb{N}$, instance size $n \in \mathbb{N}$, instance $\mathbb{x} \in \{0, 1\}^n$, and adversary $\mathcal{A}$,*

$$\Pr \left[\begin{array}{l} d = 0 \\ \wedge (\mathbb{x}, \mathbb{w}) \in R_f \end{array} \mid \begin{array}{l} f \leftarrow \mathcal{O}_{\lambda, n} \\ \mathbb{w} \leftarrow \mathcal{A}^f(\mathbb{x}) \\ d \leftarrow \langle \mathcal{P}^f(1^\lambda, (\mathbb{x}, \mathbb{w})), \mathcal{V}^f(1^\lambda, \mathbb{x}) \rangle \end{array} \right] \leq \alpha(\lambda, n) .$$

Definition 3.14 (Soundness). *$\text{ARG} = (\mathcal{P}, \mathcal{V})$ for a relativized relation $R_{\mathcal{O}}$ has **soundness error** β if for every security parameter $\lambda \in \mathbb{N}$, instance size bound $n \in \mathbb{N}$, instance $\mathbb{x} \in \{0, 1\}^n$, adversary query bound $q_{\tilde{\mathcal{P}}} \in \mathbb{N}$, adversary time bound $t_{\tilde{\mathcal{P}}} \in \mathbb{N}$, and $q_{\tilde{\mathcal{P}}}$ -query $t_{\tilde{\mathcal{P}}}$ -time adversary $\tilde{\mathcal{P}}$,*

$$\Pr \left[\begin{array}{l} d = 1 \\ \wedge \mathbb{x} \notin L(R_f) \end{array} \mid \begin{array}{l} f \leftarrow \mathcal{O}_{\lambda, n} \\ d \leftarrow \langle \tilde{\mathcal{P}}^f(\mathbb{x}), \mathcal{V}^f(1^\lambda, \mathbb{x}) \rangle \end{array} \right] \leq \beta(\lambda, n, q_{\tilde{\mathcal{P}}}, t_{\tilde{\mathcal{P}}}) .$$

Efficiency measures. We consider the following efficiency measures of ARG , each a function of the security parameter λ and the instance size n : the round complexity k ; the prover communication complexity pc , in bits; the query complexity vq and the running time vt of the verifier; the query complexity pq and the running time pt of the honest prover; and the randomness complexities vr and pr of the verifier and the honest prover.

Definition 3.15. *Let $\text{ARG} = (\mathcal{P}, \mathcal{V})$ be a relativized interactive argument for a relativized relation $R_{\mathcal{O}}$. ARG has **witness-hiding error** ε_{WH} if for every security parameter $\lambda \in \mathbb{N}$, instance size bound $n \in \mathbb{N}$, instance $\mathbb{x} \in \{0, 1\}^n$, adversary $\mathcal{A}_{\text{WH}, 1}$, and $\text{aq}(\lambda, n)$ -query $\text{at}(\lambda, n)$ -time adversary $\mathcal{A}_{\text{WH}, 2}$,*

$$\Pr \left[\begin{array}{l} (\mathbb{x}, \mathbb{w}') \in R_f \wedge (\mathbb{x}, \mathbb{w}) \in R_f \end{array} \mid \begin{array}{l} f \leftarrow \mathcal{O}_{\lambda, n} \\ \mathbb{w} \leftarrow \mathcal{A}_{\text{WH}, 1}^f(\mathbb{x}) \\ d \xleftarrow{\zeta, \rho, \text{tr}} \langle \mathcal{P}^f(1^\lambda, (\mathbb{x}, \mathbb{w})), \mathcal{V}^f(1^\lambda, \mathbb{x}) \rangle \\ \mathbb{w}' \leftarrow \mathcal{A}_{\text{WH}, 2}^f(\mathbb{x}, \rho, \text{tr}) \end{array} \right] \leq \varepsilon_{\text{WH}}(\lambda, n, \text{aq}, \text{at}) .$$

Definition 3.16 (Black-box rewinding access). For a probabilistic interactive oracle algorithm $\mathcal{A}$ and a fixed oracle function f , we write $\mathcal{B}^{f,\mathcal{A}}$ to denote that $\mathcal{B}$ has oracle access to f and **black-box rewinding access** to $\mathcal{A}$ relative to f : $\mathcal{B}$ can start executions of $\mathcal{A}$, each with an independently sampled random tape, send messages to $\mathcal{A}$ to obtain its next message, rewind $\mathcal{A}$ to an earlier point of an execution (with the same random tape), and obtain the view of $\mathcal{A}$ in a completed execution (Definition 3.9). The oracle queries of $\mathcal{A}$ are answered by f , and they count toward the running time and query complexity of $\mathcal{B}$.

Definition 3.17 (Knowledge soundness). $\text{ARG} = (\mathcal{P}, \mathcal{V})$ for a relativized relation $R_{\mathcal{O}}$ has **knowledge soundness error** κ **with extraction time** et **and query complexity** eq if there exists a probabilistic algorithm $\mathcal{E}$ such that for every security parameter $\lambda \in \mathbb{N}$, instance size bound $n \in \mathbb{N}$, oracle function $f \in \mathcal{O}_{\lambda,n}$, instance $\mathbb{x} \in \{0, 1\}^n$, adversary query bound $\text{aq} \in \mathbb{N}$, adversary time bound $\text{at} \in \mathbb{N}$, and aq -query at -time malicious prover $\tilde{\mathcal{P}}$,

$$\Pr \left[\begin{array}{c} (\mathbb{x}, \mathbb{w}) \notin R_f \\ \wedge d = 1 \end{array} \mid \begin{array}{c} d \xleftarrow{\text{tr}} \langle \tilde{\mathcal{P}}^f(\mathbb{x}), \mathcal{V}^f(1^\lambda, \mathbb{x}) \rangle \\ \mathbb{w} \leftarrow \mathcal{E}^{f, \tilde{\mathcal{P}}(\mathbb{x})}(1^\lambda, \mathbb{x}, \text{tr}, d) \end{array} \right] \leq \kappa_f(\lambda, n, \text{aq}, \text{at}) ;$$

moreover, $\mathcal{E}$ runs in time $\text{et}(\lambda, n, \text{aq}, \text{at})$ and makes at most $\text{eq}(\lambda, n, \text{aq}, \text{at})$ queries to the oracle. Above, $\mathcal{E}^{f, \tilde{\mathcal{P}}(\mathbb{x})}$ has oracle access to f and black-box rewinding access to $\tilde{\mathcal{P}}(\mathbb{x})$ relative to f (Definition 3.16).

Definition 3.18. A relativized argument $(\mathcal{P}, \mathcal{V})$ for a relativized relation $R_{\mathcal{O}}$ has **malicious-verifier zero-knowledge error** z **with simulator time** st **and query complexity** sq if there exists an st -time sq -query simulator $\mathcal{S}$ with black-box rewinding access to the malicious verifier as in Definition 3.16 such that for every security parameter $\lambda \in \mathbb{N}$, instance size bound $n \in \mathbb{N}$, oracle function $f \in \mathcal{O}_{\lambda,n}$, instance-witness pair $(\mathbb{x}, \mathbb{w}) \in R_f$, auxiliary input $\text{aux} \in \{0, 1\}^*$, which may depend on f , verifier query bound $\text{aq} \in \mathbb{N}$, verifier time bound $\text{at} \in \mathbb{N}$, and aq -query at -time malicious verifier $\tilde{\mathcal{V}}$, the following two distributions are $z(\lambda, n, \text{aq}, \text{at})$ -close in statistical distance:

- the simulated view $\mathcal{S}^{f, \tilde{\mathcal{V}}(\mathbb{x}, \text{aux})}(1^\lambda, \mathbb{x})$;
- the real view $\text{View}_{\tilde{\mathcal{V}}}(f, (\mathbb{x}, \text{aux}), (b_i)_{i \in [k]}, r)$ of $\tilde{\mathcal{V}}$ (Definition 3.9), where r is the randomness of $\tilde{\mathcal{V}}$ and $(b_i)_{i \in [k]}$ are the messages of $\mathcal{P}$ in the interaction $\langle \mathcal{P}^f(1^\lambda, (\mathbb{x}, \mathbb{w})), \tilde{\mathcal{V}}^f(\mathbb{x}, \text{aux}) \rangle$.

Zero knowledge implies witness hiding whenever witnesses are hard to find from the instance alone.

Lemma 3.19. Let $\text{ARG} = (\mathcal{P}, \mathcal{V})$ be a relativized interactive argument for a relativized relation $R_{\mathcal{O}}$ with malicious-verifier zero-knowledge error z , simulator time st , and query complexity sq (Definition 3.18). Suppose that for every security parameter $\lambda \in \mathbb{N}$, instance size bound $n \in \mathbb{N}$, instance $\mathbb{x} \in \{0, 1\}^n$, and q -query t -time algorithm $\mathcal{B}$,

$$\Pr \left[(\mathbb{x}, \mathbb{w}') \in R_f \mid \begin{array}{c} f \leftarrow \mathcal{O}_{\lambda,n} \\ \mathbb{w}' \leftarrow \mathcal{B}^f(\mathbb{x}) \end{array} \right] \leq \varepsilon_{\text{find}}(\lambda, n, q, t) .$$

Then ARG has witness-hiding error ε_{WH} (Definition 3.15) such that

$$\varepsilon_{\text{WH}}(\lambda, n, \text{aq}, \text{at}) \leq z(\lambda, n, \text{vq}, \text{vt}) + \varepsilon_{\text{find}}(\lambda, n, \text{sq}(\lambda, n, \text{vq}, \text{vt}) + \text{aq}, O(\text{st}(\lambda, n, \text{vq}, \text{vt}) + \text{at})) .$$

Proof. Fix $\lambda, n \in \mathbb{N}$, an instance $\mathbb{x} \in \{0, 1\}^n$, an adversary $\mathcal{A}_{\text{WH},1}$, and a aq -query at -time adversary $\mathcal{A}_{\text{WH},2}$. Let $\mathcal{S}$ be the simulator of Definition 3.18, and let $\mathcal{B}$ be the following algorithm. Given $\mathbb{x}$ and oracle access to f , it runs $\mathcal{S}^{f, \mathcal{V}}(1^\lambda, \mathbb{x})$ for the honest verifier $\mathcal{V}$ with an empty auxiliary input, answering the oracle queries of

$\mathcal{V}$ with f . It then computes the verifier randomness ρ and the transcript tr from the simulated view, which determines both, and outputs $\mathcal{A}_{\text{WH},2}^f(\mathbb{x}, \rho, \text{tr})$. The algorithm $\mathcal{B}$ makes at most $\text{sq} + \text{aq}$ queries and runs in time $O(\text{st} + \text{at})$.

Fix $f \in \mathcal{O}_{\lambda,n}$ and $\mathbb{w}$ such that $(\mathbb{x}, \mathbb{w}) \in R_f$. By Definition 3.18, the simulated view is z -close in statistical distance to the view of $\mathcal{V}$ in its interaction with $\mathcal{P}^f(1^\lambda, (\mathbb{x}, \mathbb{w}))$. Statistical distance does not increase when we apply $\mathcal{A}_{\text{WH},2}^f$ and test membership in R_f . Hence the probability that $\mathcal{A}_{\text{WH},2}$ outputs a valid witness after the real interaction is at most z plus the probability that $\mathcal{B}^f(\mathbb{x})$ outputs a valid witness. Averaging over $f \leftarrow \mathcal{O}_{\lambda,n}$ and $\mathbb{w} \leftarrow \mathcal{A}_{\text{WH},1}^f(\mathbb{x})$ bounds the probability in Definition 3.15 by $z + \varepsilon_{\text{find}}$ at the stated resources. $\square$

Definition 3.20 (Relativized ARG in an oracle model). *Let $\mathcal{O}$ be an oracle. The complexity class*

$$\text{ARG}^{\mathcal{O}} \left[\begin{array}{lll} \text{completeness error} & \alpha & = \alpha(\lambda, n) \\ \text{soundness error} & \beta & = \beta(\lambda, n, \text{q}_{\tilde{\mathcal{P}}}, \text{t}_{\tilde{\mathcal{P}}}) \\ \text{knowledge soundness error} & \kappa & = \kappa(\lambda, n, \text{aq}, \text{at}) \\ \text{witness-hiding error} & \varepsilon_{\text{WH}} & = \varepsilon_{\text{WH}}(\lambda, n, \text{aq}, \text{at}) \\ \text{zero-knowledge error} & z & = z(\lambda, n, \text{aq}, \text{at}) \\ \text{verifier query bound} & \text{vq} & = \text{vq}(\lambda, n) \\ \text{verifier running time} & \text{vt} & = \text{vt}(\lambda, n) \\ \text{prover communication} & \text{pc} & = \text{pc}(\lambda, n) \\ \text{honest prover query bound} & \text{pq} & = \text{pq}(\lambda, n) \\ \text{honest prover time bound} & \text{pt} & = \text{pt}(\lambda, n) \end{array} \right]$$

is the set of relativized relations $R_{\mathcal{O}} = \{R_f\}_{\lambda, n \in \mathbb{N}, f \in \mathcal{O}_{\lambda,n}}$ that admit a relativized interactive argument $(\mathcal{P}, \mathcal{V})$ with the listed errors and efficiency measures. A parameter that is omitted is unconstrained.

Arguments for non-relativized relations. A relation R that does not depend on the oracle is the relativized relation with $R_f = R$ for every $f \in \mathcal{O}$. When, in addition, the honest prover and verifier make no oracle queries, we call ARG an *interactive argument for* R and say that its knowledge soundness error and zero-knowledge error hold *relative to* $\mathcal{O}$: the oracle is available only to the adversary, the extractor, and the simulator.

3.4 Collision-resistant hash functions

A keyed hash family is defined as follows.

Definition 3.21. A **keyed hash family** is a pair of polynomial-time algorithms $\text{H} = (\text{H.Gen}, \text{H})$, where $\text{H.Gen}(1^\lambda)$ outputs a key hk and $H_{\text{hk}}: \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$ is a deterministic function. It has **collision-resistance error** ε_{cr} **relative to** $\mathcal{O}$ if for every $\lambda, n \in \mathbb{N}$, $f \in \mathcal{O}_{\lambda,n}$, and t -time adversary $\mathcal{A}^f$ making at most q queries to f ,

$$\Pr \left[H_{\text{hk}}(m) = H_{\text{hk}}(m') \wedge m \neq m' \mid \begin{array}{l} \text{hk} \leftarrow \text{H.Gen}(1^\lambda) \\ (m, m') \leftarrow \mathcal{A}^f(1^\lambda, \text{hk}) \end{array} \right] \leq \varepsilon_{\text{cr}}^f(\lambda, \text{q}, \text{t}) .$$

The family is **collision resistant relative to** $\mathcal{O}$ if for every $f \in \mathcal{O}_{\lambda,n}$, $\varepsilon_{\text{cr}}^f(\lambda, \text{q}, \text{t})$ is negligible whenever n, q, t are polynomially bounded in λ .

3.5 Digital signatures

A digital signature scheme is defined as follows.

Definition 3.22. A **digital signature scheme** for messages in $\{0, 1\}^*$ is a tuple of polynomial-time algorithms $\text{SIG} = (\text{Gen}, \text{Sign}, \text{Verify})$ with the following syntax.

- $\text{Gen}(1^\lambda)$ outputs a signing key sk and a verification key vk .
- $\text{Sign}(\text{sk}, m)$ outputs a signature σ on a message m .
- $\text{Verify}(\text{vk}, m, \sigma)$ deterministically outputs a bit indicating whether σ is a valid signature on m under vk .

The scheme SIG is a digital signature scheme with unforgeability error ε_{sig} relative to an oracle $\mathcal{O}$ if the following hold.

- **Perfect correctness.** For every $\lambda \in \mathbb{N}$, every $(\text{sk}, \text{vk}) \in \text{Supp}(\text{Gen}(1^\lambda))$, and every message m ,

$$\Pr [\text{Verify}(\text{vk}, m, \sigma) = 1 \mid \sigma \leftarrow \text{Sign}(\text{sk}, m)] = 1 .$$

- **Existential unforgeability under adaptive chosen-message attacks (EUF-CMA) error ε_{sig} relative to $\mathcal{O}$.** For every $\lambda, n \in \mathbb{N}$, $f \in \mathcal{O}_{\lambda, n}$, and t -time adversary $\mathcal{A}^f$ making at most q queries in total to f and to the adaptive signing oracle $\text{Sign}(\text{sk}, \cdot)$,

$$\Pr \left[\text{Verify}(\text{vk}, m^*, \sigma^*) = 1 \wedge m^* \notin Q \mid \begin{array}{l} (\text{sk}, \text{vk}) \leftarrow \text{Gen}(1^\lambda) \\ (m^*, \sigma^*) \leftarrow \mathcal{A}^{f, \text{Sign}(\text{sk}, \cdot)}(1^\lambda, \text{vk}) \end{array} \right] \leq \varepsilon_{\text{sig}}^f(\lambda, q, t) ,$$

where Q is the set of messages submitted to the signing oracle $\text{Sign}(\text{sk}, \cdot)$. The scheme is **EUF-CMA secure relative to $\mathcal{O}$** if for every $f \in \mathcal{O}_{\lambda, n}$, $\varepsilon_{\text{sig}}^f(\lambda, q, t)$ is negligible whenever n, q, t are polynomially bounded in λ .

Theorem 3.23 ([NY89; Rom90; KK05]). Assuming the existence of one-way functions (OWF), there exists an EUF-CMA secure digital signature scheme (relative to the empty oracle).

Collision-resistant hash functions imply one-way functions, and both constructions are black-box, so the theorem extends directly to any oracle.

Corollary 3.24. Let $\mathcal{O}$ be an oracle. If there exists a keyed hash family that is collision resistant relative to $\mathcal{O}$ (Definition 3.21), then there exists a digital signature scheme that is EUF-CMA secure relative to $\mathcal{O}$.

3.6 Probabilistically checkable proofs

A **probabilistically checkable proof (PCP)** for a relation R is a pair of probabilistic algorithms $\text{PCP} = (\mathcal{P}_{\text{PCP}}, \mathcal{V}_{\text{PCP}})$ that satisfies the following properties. The prover $\mathcal{P}_{\text{PCP}}(\mathbb{x}, \mathbb{w})$ outputs a proof string $\pi \in \{0, 1\}^\ell$. The verifier $\mathcal{V}_{\text{PCP}}^\pi(\mathbb{x}; r)$ receives the instance $\mathbb{x}$ and randomness $r \in \{0, 1\}^r$, has oracle access to π , and outputs a decision bit; each query is an index $i \in [\ell]$ answered by the bit $\pi[i]$. The **view** of the verifier in the execution $\mathcal{V}_{\text{PCP}}^\pi(\mathbb{x}; r)$ is $(r, I, \pi|_I)$, where $I \subseteq [\ell]$ is the set of indices that it queries. The PCP has proof length ℓ , query complexity q_{PCP} , randomness complexity r in bits, and verifier running time vt_{PCP} ; these measures may depend on the instance $\mathbb{x}$, for example on $|\mathbb{x}|$.

Definition 3.25 (Completeness). $\text{PCP} = (\mathcal{P}_{\text{PCP}}, \mathcal{V}_{\text{PCP}})$ for a relation R has **completeness error** γ_{PCP} if, for every instance size bound $n \in \mathbb{N}$ and every instance-witness pair $(\mathbb{x}, \mathbb{w}) \in R$ with $|\mathbb{x}| \leq n$,

$$\Pr \left[\mathcal{V}_{\text{PCP}}^\pi(\mathbb{x}; r) = 1 \mid \begin{array}{l} \pi \leftarrow \mathcal{P}_{\text{PCP}}(\mathbb{x}, \mathbb{w}) \\ r \leftarrow \{0, 1\}^r \end{array} \right] \geq 1 - \gamma_{\text{PCP}}(n) .$$

It has **perfect completeness** if γ_{PCP} is zero.

Definition 3.26 (Knowledge soundness). $\text{PCP} = (\mathcal{P}_{\text{PCP}}, \mathcal{V}_{\text{PCP}})$ for a relation R has **knowledge soundness error** κ_{PCP} **with extraction time** t_E if there exists a probabilistic algorithm $\mathcal{E}_{\text{PCP}}$ such that, for every instance size bound $n \in \mathbb{N}$, instance $\mathbb{x}$ with $|\mathbb{x}| \leq n$, and proof string $\pi \in \{0, 1\}^\ell$,

$$\Pr \left[\begin{array}{c|c} (\mathbb{x}, \mathbb{w}) \notin R & r \leftarrow \{0, 1\}^r \\ \wedge \mathcal{V}_{\text{PCP}}^\pi(\mathbb{x}; r) = 1 & \mathbb{w} \leftarrow \mathcal{E}_{\text{PCP}}(\mathbb{x}, \pi) \end{array} \right] \leq \kappa_{\text{PCP}}(n) ;$$

moreover, $\mathcal{E}_{\text{PCP}}$ runs in time $t_E(n)$.

Definition 3.27 (Perfect honest-verifier zero knowledge). A PCP $\text{PCP} = (\mathcal{P}_{\text{PCP}}, \mathcal{V}_{\text{PCP}})$ is **perfect honest-verifier zero knowledge (HVZK)** if there exists a probabilistic algorithm $\mathcal{S}_{\text{PCP}}$ such that for every $(\mathbb{x}, \mathbb{w}) \in R$, the following two random variables are identically distributed:

$$\mathcal{S}_{\text{PCP}}(\mathbb{x}) \text{ and } \left\{ (r, I, \pi|_I) \left| \begin{array}{l} \pi \leftarrow \mathcal{P}_{\text{PCP}}(\mathbb{x}, \mathbb{w}) \\ r \leftarrow \{0, 1\}^r \\ I := \text{the indices queried by } \mathcal{V}_{\text{PCP}}^\pi(\mathbb{x}; r) \end{array} \right. \right\}. \quad (2)$$

Moreover, $\mathcal{S}_{\text{PCP}}$ runs in time at most $\text{poly}(|\mathbb{x}|, \text{vt}_{\text{PCP}})$.

Theorem 3.28. *There is a perfect honest-verifier zero-knowledge PCP (Definition 3.27) for the universal relation R_U (Definition 3.12) with perfect completeness, knowledge soundness error at most $1/2$, constant query complexity, proof length $\text{poly}(t)$, randomness complexity $O(\log t)$, and verifier running time $\text{poly}(|M| + |\mathbb{x}| + \log t)$. Its prover and extractor run in time $\text{poly}(|M| + |\mathbb{x}| + t)$.*

Proof. Apply the honest-verifier zero-knowledge transformation of [IW14, Theorem 5], whose composition step preserves knowledge soundness [CF24, Lemma A.5], to a constant number of parallel repetitions of the PCP of knowledge for R_U of [BCCT12, Section 3.4] (obtained from [BGHSV05] via [Val08]); sequential repetition on independent proofs preserves honest-verifier zero knowledge and reduces the knowledge soundness error. $\square$

3.7 Vector commitment schemes

A **vector commitment scheme** is a tuple of polynomial-time algorithms

$$\text{VC} = (\text{VC.Gen}, \text{VC.Commit}, \text{VC.Open}, \text{VC.Check})$$

with the following syntax.

- $\text{VC.Gen}(1^\lambda, N)$, on input a security parameter $\lambda \in \mathbb{N}$ and a message length $N \in \mathbb{N}$, samples public parameters pp_{VC} .
- $\text{VC.Commit}(\text{pp}_{\text{VC}}, \mathbf{m})$, on input public parameters pp_{VC} and a message $\mathbf{m} \in \{0, 1\}^N$, outputs a commitment cm and an auxiliary state aux .
- $\text{VC.Open}(\text{pp}_{\text{VC}}, \text{aux}, Q)$, on input public parameters pp_{VC} , an auxiliary state aux , and a query set $Q \subseteq [N]$, outputs an opening proof pf for $\mathbf{m}[Q]$.
- $\text{VC.Check}(\text{pp}_{\text{VC}}, \text{cm}, Q, \text{ans}, \text{pf})$, on input public parameters pp_{VC} , a commitment cm , a query set $Q \subseteq [N]$, an answer string $\text{ans} \in \{0, 1\}^Q$, and an opening proof pf , outputs a bit indicating whether pf is a valid proof that ans is the restriction to Q of the message committed in cm .

Since in our protocols the verifier samples the public parameters (Section 3.3), we assume that membership in the support of $\text{VC.Gen}(1^\lambda, N)$ can be decided in time $\text{poly}(\lambda, N)$.

Definition 3.29 (Completeness). $\text{VC} = (\text{VC.Gen}, \text{VC.Commit}, \text{VC.Open}, \text{VC.Check})$ has **perfect completeness** if for every security parameter $\lambda \in \mathbb{N}$, message length $N \in \mathbb{N}$, message $\mathbf{m} \in \{0, 1\}^N$, and query set $Q \subseteq [N]$,

$$\Pr \left[\text{VC.Check}(\text{pp}_{\text{vc}}, \text{cm}, Q, \mathbf{m}[Q], \text{pf}) = 1 \mid \begin{array}{l} \text{pp}_{\text{vc}} \leftarrow \text{VC.Gen}(1^\lambda, N) \\ (\text{cm}, \text{aux}) \leftarrow \text{VC.Commit}(\text{pp}_{\text{vc}}, \mathbf{m}) \\ \text{pf} \leftarrow \text{VC.Open}(\text{pp}_{\text{vc}}, \text{aux}, Q) \end{array} \right] = 1 .$$

Definition 3.30 (Position binding). $\text{VC} = (\text{VC.Gen}, \text{VC.Commit}, \text{VC.Open}, \text{VC.Check})$ has **position binding error** β_{vc} **relative to** $\mathcal{O}$ if for every security parameter $\lambda \in \mathbb{N}$, instance size bound $n \in \mathbb{N}$, oracle function $f \in \mathcal{O}_{\lambda, n}$, message length $N \in \mathbb{N}$, query set size $s \in \mathbb{N}$ with $s \leq N$, and q -query t -time adversary $\mathcal{A}$,

$$\Pr \left[\begin{array}{l} |Q| = |Q'| = s \\ \wedge \exists i \in Q \cap Q' : \text{ans}[i] \neq \text{ans}'[i] \\ \wedge \text{VC.Check}(\text{pp}_{\text{vc}}, \text{cm}, Q, \text{ans}, \text{pf}) = 1 \\ \wedge \text{VC.Check}(\text{pp}_{\text{vc}}, \text{cm}, Q', \text{ans}', \text{pf}') = 1 \end{array} \mid \begin{array}{l} \text{pp}_{\text{vc}} \leftarrow \text{VC.Gen}(1^\lambda, N) \\ (\text{cm}, Q, Q', \text{ans}, \text{ans}', \text{pf}, \text{pf}') \leftarrow \mathcal{A}^f(1^\lambda, \text{pp}_{\text{vc}}) \end{array} \right] \leq \beta_{\text{vc}}^f(\lambda, N, s, q, t) .$$

It is **position binding relative to** $\mathcal{O}$ if for every $f \in \mathcal{O}_{\lambda, n}$, $\beta_{\text{vc}}^f(\lambda, N, s, q, t)$ is negligible whenever n, N, s, q, t are polynomially bounded in λ .

Definition 3.31 (Hiding). $\text{VC} = (\text{VC.Gen}, \text{VC.Commit}, \text{VC.Open}, \text{VC.Check})$ has **hiding error** η_{vc} if for every security parameter $\lambda \in \mathbb{N}$, message length $N \in \mathbb{N}$, public parameters $\text{pp}_{\text{vc}} \in \text{Supp}(\text{VC.Gen}(1^\lambda, N))$, query set $Q \subseteq [N]$, and messages $\mathbf{m}^0, \mathbf{m}^1 \in \{0, 1\}^N$ with $\mathbf{m}^0[Q] = \mathbf{m}^1[Q]$, the following two random variables are $\eta_{\text{vc}}(\lambda, N)$ -close in statistical distance:

- $\left\{ (\text{cm}, \text{pf}) \mid \begin{array}{l} (\text{cm}, \text{aux}) \leftarrow \text{VC.Commit}(\text{pp}_{\text{vc}}, \mathbf{m}^0) \\ \text{pf} \leftarrow \text{VC.Open}(\text{pp}_{\text{vc}}, \text{aux}, Q) \end{array} \right\};$
- $\left\{ (\text{cm}, \text{pf}) \mid \begin{array}{l} (\text{cm}, \text{aux}) \leftarrow \text{VC.Commit}(\text{pp}_{\text{vc}}, \mathbf{m}^1) \\ \text{pf} \leftarrow \text{VC.Open}(\text{pp}_{\text{vc}}, \text{aux}, Q) \end{array} \right\}.$

Definition 3.32. $\text{VC} = (\text{VC.Gen}, \text{VC.Commit}, \text{VC.Open}, \text{VC.Check})$ is **succinct** if the running time of VC.Gen and VC.Check and the lengths of pp_{vc} , cm , and pf are $\text{poly}(\lambda, |Q|, \log N)$, and VC.Commit and VC.Open run in time $\text{poly}(\lambda, N)$.

Theorem 3.33. Let $\mathcal{O}$ be an oracle. If there exists a keyed hash family with collision-resistance error ε_{cr} relative to $\mathcal{O}$ (Definition 3.21), then there exists a succinct vector commitment scheme with perfect completeness, hiding error $\eta_{\text{vc}}(\lambda, N) \leq 2^{-\lambda}$, and position binding error

$$\beta_{\text{vc}}^f(\lambda, N, s, q, t) \leq \varepsilon_{\text{cr}}^f(\lambda, q, t + \text{poly}(\lambda, s, \log N))$$

relative to $\mathcal{O}$; in particular, it is position binding relative to $\mathcal{O}$ if the family is collision resistant relative to $\mathcal{O}$.

Proof. Commit to each bit of the message with the statistically hiding commitment of Halevi–Micali [HM96], with hiding parameter $\lambda + \lceil \log N \rceil$, and place the N commitments at the leaves of a Merkle tree [Mer89] built from the hash family: two accepting openings that disagree at a position yield a collision in the hash family, and a hybrid over the leaves bounds the hiding error by $N \cdot 2^{-\lambda - \lceil \log N \rceil} \leq 2^{-\lambda}$. $\square$

4 Impossibility of witness hiding in the random oracle model

We prove that there are no witness-hiding relativized arguments for $\text{NTIME}^{\mathcal{U}}$:

Theorem 4.1. *Let $t: \mathbb{N} \rightarrow \mathbb{N}$ be a time-constructible function with $t(n) \geq n$ and $\mathcal{U}$ be the random oracle over $((\{0, 1\}^{t(n)+\lceil \log t(n) \rceil + 1}, \{0, 1\}^\lambda))_{\lambda, n \in \mathbb{N}}$.*

$$\text{NTIME}^{\mathcal{U}}[t] \not\subseteq \text{ARG}^{\mathcal{U}} \left[\begin{array}{ll} \text{completeness error} & \alpha = 1/10 \\ \text{soundness error} & \beta = \begin{cases} 1/10 & \text{q}_{\tilde{\mathcal{P}}} = \text{poly}(\lambda, t(n)) \\ \text{t}_{\tilde{\mathcal{P}}} = \text{poly}(\lambda, t(n)) & \\ 1 & \text{otherwise} \end{cases} \\ \text{verifier query bound} & \text{vq} = \text{poly}(\lambda, t(n)) \\ \text{verifier running time} & \text{vt} = \text{poly}(\lambda, t(n)) \\ \text{honest prover query bound} & \text{pq} = \text{poly}(\lambda, t(n)) \\ \text{honest prover time bound} & \text{pt} = \text{poly}(\lambda, t(n)) \\ \text{witness-hiding error} & \varepsilon_{\text{WH}} = \begin{cases} 1/10 & \text{aq} = \text{poly}(\lambda, t(n)) \\ \text{at} = \text{poly}(\lambda, t(n)) & \\ 1 & \text{otherwise} \end{cases} \end{array} \right].$$

In order to prove Theorem 4.1, we prove that there are no witness-hiding relativized arguments for the relativized relation of [BCG25], which we recall next.

Definition 4.2 ([BCG25, Definition 5.3]). *Let $t: \mathbb{N} \rightarrow \mathbb{N}$ be a time-constructible function with $t(n) \geq n$ for every $n \in \mathbb{N}$ and $\mathcal{U}$ be the random oracle over $((\{0, 1\}^{t(n)+\lceil \log t(n) \rceil + 1}, \{0, 1\}^\lambda))_{\lambda, n \in \mathbb{N}}$. Let $e_{k,i}$ denote the $(\lceil \log k \rceil + 1)$ -bit string that represents $i \in [k]$. We define $R_{\mathcal{U}} = \{R_u\}_{\lambda, n \in \mathbb{N}, u \in \mathcal{U}_{\lambda, n}}$ as follows.*

$$R_u := \left\{ (\mathbb{x}, \mathbb{w}) \in \{0, 1\}^n \times \{0, 1\}^{t(n)} \mid \begin{array}{l} \mathbb{x} = 0^n \\ \wedge \forall i \in [t(n)], u(\mathbb{w} \| e_{t(n), i})_1 = 0 \end{array} \right\}.$$

Claim 4.3 ([BCG25, Lemma 5.4]).

$$R_{\mathcal{U}} \in \text{NTIME}^{\mathcal{U}}[t].$$

Theorem 4.1 follows from Claim 4.3 and the following lemma since for every sufficiently large n ,

$$\left(1 - \frac{1}{2^{t(n)}}\right)^{2^{t(n)}} > 0.36.$$

Lemma 4.4. *Let $t: \mathbb{N} \rightarrow \mathbb{N}$ be a time-constructible function with $t(n) \geq n$ for every $n \in \mathbb{N}$ and $\mathcal{U}$ be the random oracle over $((\{0, 1\}^{t(n)+\lceil \log t(n) \rceil + 1}, \{0, 1\}^\lambda))_{\lambda, n \in \mathbb{N}}$. Let $R_{\mathcal{U}}$ be as in Definition 4.2. If $\text{ARG} = (\mathcal{P}, \mathcal{V})$ is a relativized argument for $R_{\mathcal{U}}$, then the efficiency parameters of ARG satisfy*

$$\alpha + \beta(\text{pq}, O(\text{pt} + \text{pq} \cdot t(n))) + \varepsilon_{\text{WH}}(O(\text{vq} \cdot t(n)), O(\text{vt} + \text{vq} \cdot t(n))) \geq \left(1 - \frac{1}{2^{t(n)}}\right)^{2^{t(n)}}, \quad (3)$$

where we suppress the common security parameter λ and instance size n for simplicity.

We fix $\lambda, n \in \mathbb{N}$ and suppress them throughout the rest of this section. The proof of Lemma 4.4 relies on the fact that we can switch between completeness and soundness cases by modifying the oracle at a few locations. In Section 4.1, we define how we modify the oracle to insert a witness and show how this modification changes the probability distributions. In Section 4.2, we prove Lemma 4.4.

4.1 How to modify the oracle to insert a witness

We define a function `Insert` to modify the oracle to insert a witness as follows. For $u \in \mathcal{U}_{\lambda,n}$, $\mathbb{w} \in \{0,1\}^{t(n)}$, `Insert`($u, \mathbb{w}$):

1. Initialize $u' := u$.
2. For all $i \in [t(n)]$: $u'(\mathbb{w} \| e_{t(n),i})_1 := 0$.
3. Output u' .

We note that `Insert` is identical to how [BCG25] modifies the oracle and the following claims are implicit in [BCG25].

Lemma 4.5. *For every set Z , predicate ϕ over $\mathcal{U}_{\lambda,n} \times (\{0,1\}^{t(n)} \cup \{\perp\}) \times Z$, and family of distributions $\{\mathcal{D}_{(u_{(0)}, \mathbb{w}_0)}\}_{(u_{(0)}, \mathbb{w}_0) \in \mathcal{U}_{\lambda,n} \times (\{0,1\}^{t(n)} \cup \{\perp\})}$ over Z ,*

$$\Pr_{\text{EXP}} [\phi(u_{(0)}, \mathbb{w}_0, z) = 1 \wedge 0^n \notin L(R_u)] = \left(1 - \frac{1}{2^{t(n)}}\right) \cdot \Pr_{\text{EXP}'} [\phi(u, \mathbb{w}_0, z) = 1 \wedge (0^n, \mathbb{w}_0) \in R_u] ,$$

where $\mathcal{A}$ is defined as follows.

$\mathcal{A}^u$:

1. Compute $\text{Witness}^{Ru}(u, 0^n)$ by querying u as needed.
2. If there exists some $\mathbb{w}$ such that $\text{Witness}^{Ru}(u, 0^n) = \{\mathbb{w}\}$, output $\mathbb{w}$. Otherwise, output $\perp$.

and

$$\text{EXP} := \left\{ \begin{array}{l} u \leftarrow \mathcal{U}_{\lambda,n} \\ \mathbb{w}_0 \leftarrow \{0,1\}^{t(n)} \\ u_{(0)} := \text{Insert}(u, \mathbb{w}_0) \\ z \leftarrow \mathcal{D}_{(u_{(0)}, \mathbb{w}_0)} \end{array} \right\} \quad \text{and} \quad \text{EXP}' := \left\{ \begin{array}{l} u \leftarrow \mathcal{U}_{\lambda,n} \\ \mathbb{w}_0 \leftarrow \mathcal{A}^u \\ z \leftarrow \mathcal{D}_{(u, \mathbb{w}_0)} \end{array} \right\} .$$

Proof. We have

$$\begin{aligned} \Pr_{\text{EXP}} [\phi(u_{(0)}, \mathbb{w}_0, z) = 1 \wedge 0^n \notin L(R_u)] &= \Pr_{\text{EXP}} [\phi(u_{(0)}, \mathbb{w}_0, z) = 1 \wedge 0^n \notin L(R_u)] \\ &\quad - \Pr_{\text{EXP}} \left[\begin{array}{l} \phi(u_{(0)}, \mathbb{w}_0, z) = 1 \wedge 0^n \notin L(R_u) \\ \wedge \text{Witness}^{Ru}(u_{(0)}, 0^n) \neq \{\mathbb{w}_0\} \end{array} \right] \\ &\quad \text{(since Insert adds exactly } \mathbb{w} \text{ to the witness set by definition)} \\ &= \Pr_{\text{EXP}} \left[\begin{array}{l} \phi(u_{(0)}, \mathbb{w}_0, z) = 1 \wedge 0^n \notin L(R_u) \\ \wedge \text{Witness}^{Ru}(u_{(0)}, 0^n) = \{\mathbb{w}_0\} \end{array} \right] . \\ &\quad \text{(by the law of total probability)} \end{aligned}$$

Therefore, we are left to prove

$$\Pr_{\text{EXP}} \left[\begin{array}{l} \phi(u_{(0)}, \mathbb{w}_0, z) = 1 \wedge 0^n \notin L(R_u) \\ \wedge \text{Witness}^{Ru}(u_{(0)}, 0^n) = \{\mathbb{w}_0\} \end{array} \right] = \left(1 - \frac{1}{2^{t(n)}}\right) \cdot \Pr_{\text{EXP}'} \left[\begin{array}{l} \phi(u, \mathbb{w}_0, z) = 1 \\ \wedge (0^n, \mathbb{w}_0) \in R_u \end{array} \right] . \quad (4)$$

By the law of total probability and chain rule, the left-hand side of Equation 4 equals

$$\sum_{\substack{u_{(0)}^* \in \mathcal{U}_{\lambda,n}, \mathbb{w}_0^* \in \{0,1\}^{t(n)} \\ \text{s.t. } \text{Witness}^{Ru}(u_{(0)}^*, 0^n) = \{\mathbb{w}_0^*\}}} \Pr_{\text{EXP}} \left[\begin{array}{l} u_{(0)} = u_{(0)}^* \\ \wedge \mathbb{w}_0 = \mathbb{w}_0^* \\ \wedge 0^n \notin L(R_u) \end{array} \right] \cdot \Pr \left[\phi(u_{(0)}^*, \mathbb{w}_0^*, z) = 1 \mid z \leftarrow \mathcal{D}_{(u_{(0)}^*, \mathbb{w}_0^*)} \right] ,$$

while the right-hand side of Equation 4 equals

$$\sum_{\substack{u_{(0)}^* \in \mathcal{U}_{\lambda,n}, \mathbb{w}_0^* \in \{0,1\}^{t(n)} \\ \text{s.t. } \text{Witness}^{Ru}(u_{(0)}^*, 0^n) = \{\mathbb{w}_0^*\}}} \left(1 - \frac{1}{2^{t(n)}}\right) \cdot \Pr_{\text{EXP}'} \left[\begin{array}{l} u = u_{(0)}^* \\ \wedge \mathbb{w}_0 = \mathbb{w}_0^* \\ \wedge (0^n, \mathbb{w}_0) \in R_u \end{array} \right] \cdot \Pr \left[\phi(u_{(0)}^*, \mathbb{w}_0^*, z) = 1 \mid z \leftarrow \mathcal{D}_{(u_{(0)}^*, \mathbb{w}_0^*)} \right] .$$

Consequently, proving that for every $\mathbb{w}_0^* \in \{0,1\}^{t(n)}$, $u_{(0)}^* \in \mathcal{U}_{\lambda,n}$ such that $\text{Witness}^{Ru}(u_{(0)}^*, 0^n) = \{\mathbb{w}_0^*\}$,

$$\Pr_{\text{EXP}} \left[\begin{array}{l} u_{(0)} = u_{(0)}^* \wedge \mathbb{w}_0 = \mathbb{w}_0^* \\ \wedge 0^n \notin L(R_u) \end{array} \right] = \left(1 - \frac{1}{2^{t(n)}}\right) \cdot \Pr_{\text{EXP}'} \left[\begin{array}{l} u = u_{(0)}^* \wedge \mathbb{w}_0 = \mathbb{w}_0^* \\ \wedge (0^n, \mathbb{w}_0) \in R_u \end{array} \right] , \quad (5)$$

proves Equation 4. In the rest, we fix $\mathbb{w}_0^* \in \{0,1\}^{t(n)}$, $u_{(0)}^* \in \mathcal{U}_{\lambda,n}$ such that $\text{Witness}^{Ru}(u_{(0)}^*, 0^n) = \{\mathbb{w}_0^*\}$ and prove Equation 5. Let

$$S := \{u^* \in \mathcal{U}_{\lambda,n} : u_{(0)}^* = \text{Insert}(u^*, \mathbb{w}_0^*) \wedge 0^n \notin L(R_{u^*})\} .$$

We have

$$\begin{aligned} \Pr_{\text{EXP}} \left[\begin{array}{l} u_{(0)} = u_{(0)}^* \wedge \mathbb{w}_0 = \mathbb{w}_0^* \\ \wedge 0^n \notin L(R_u) \end{array} \right] &= \sum_{u^* \in S} \Pr_{\text{EXP}} \left[\begin{array}{l} u = u^* \wedge \mathbb{w}_0 = \mathbb{w}_0^* \\ \wedge 0^n \notin L(R_u) \end{array} \right] \\ &\quad \text{(by the law of total probability)} \\ &= \sum_{u^* \in S} \Pr_{\text{EXP}} [u = u^* \wedge \mathbb{w}_0 = \mathbb{w}_0^*] \quad \text{(by the definition of } S) \\ &= \sum_{u^* \in S} \Pr_{\text{EXP}} [u = u_{(0)}^* \wedge \mathbb{w}_0 = \mathbb{w}_0^*] \quad \text{(by uniformity of } \mathcal{U}) \\ &= \frac{1}{2^{t(n)}} \cdot \sum_{u^* \in S} \Pr [u = u_{(0)}^* \mid u \leftarrow \mathcal{U}_{\lambda,n}] \\ &\quad \text{(by how } u \text{ and } \mathbb{w}_0 \text{ are sampled in EXP)} \\ &= \frac{|S|}{2^{t(n)}} \cdot \Pr [u = u_{(0)}^* \mid u \leftarrow \mathcal{U}_{\lambda,n}] \quad \text{(by summing over } u^*) \\ &= \frac{|S|}{2^{t(n)}} \cdot \frac{\Pr_{\text{EXP}'} [u = u_{(0)}^* \wedge \mathbb{w}_0 = \mathbb{w}_0^*]}{\Pr [\mathbb{w}_0 = \mathbb{w}_0^* \mid \mathbb{w}_0 \leftarrow \mathcal{A}^{u_{(0)}^*}]} \quad \text{(by the chain rule)} \\ &= \frac{|S|}{2^{t(n)}} \cdot \Pr_{\text{EXP}'} \left[\begin{array}{l} u = u_{(0)}^* \wedge \mathbb{w}_0 = \mathbb{w}_0^* \\ \wedge (0^n, \mathbb{w}_0) \in R_u \end{array} \right] . \\ &\quad \text{(since } \text{Witness}^{Ru}(u_{(0)}^*, 0^n) = \{\mathbb{w}_0^*\}) \end{aligned}$$

Finally, we have $|S| = 2^{t(n)} - 1$ because

$$|S| = \left| \{\text{Recover}(u_{(0)}^*, \mathbb{w}_0^*, s) : s \in \{0,1\}^{t(n)} \setminus \{0^{t(n)}\}\} \right| = 2^{t(n)} - 1 ,$$

where Recover is defined as follows for $u \in \mathcal{U}_{\lambda,n}$, $\mathbb{w} \in \{0,1\}^{t(n)}$, $s \in \{0,1\}^{t(n)}$,

Recover($u, \mathbb{w}, s$):

1. Initialize $u' := u$.
2. For all $i \in [t(n)]$: $u'(\mathbb{w} \| e_{t(n),i})_1 := s_i$.
3. Output u' .

□

4.2 Proof of Lemma 4.4

Throughout this proof, we sample the random variables according to the following random experiment.

$$\text{EXP} := \left\{ \begin{array}{l} u \leftarrow \mathcal{U}_{\lambda,n} \\ \mathbb{w}_0 \leftarrow \{0, 1\}^{t(n)} \\ u_{(0)} := \text{Insert}(u, \mathbb{w}_0) \\ \mathbf{d} \xleftarrow{\zeta, \rho, (b_1, a_1, \dots, b_k, a_k)} \langle \mathcal{P}^{u_{(0)}}(1^\lambda, (0^n, \mathbb{w}_0)), \mathcal{V}^u(1^\lambda, 0^n) \rangle \\ \mathbf{d}^{(0)} \xleftarrow{(b_1^{(0)}, a_1^{(0)}, \dots, b_k^{(0)}, a_k^{(0)})} \langle \mathcal{P}^{u_{(0)}}(1^\lambda, (0^n, \mathbb{w}_0); \zeta), \mathcal{V}^{u_{(0)}}(1^\lambda, 0^n; \rho) \rangle \end{array} \right\}.$$

To prove Equation 3, we analyze the probability that the verifier queries the oracle at a location relevant for $\mathbb{w}_0$. Formally, we define a verifier query predicate as follows: $Q_V(u, (b_i)_{i \in [k]}, \rho, \mathbb{w})$ evaluates to 1 if and only if the verifier $\mathcal{V}^u(1^\lambda, 0^n, (b_i)_{i \in [j-1]}; \rho)$ queries u at $\mathbb{w} \| e_{t(n), i}$ for some $i \in [t(n)]$ and $j \in [k+1]$.

Equation 3 follows from the following.

$$\begin{aligned} \left(1 - \frac{1}{2^{t(n)}}\right)^{2^{t(n)}} - \beta(\mathbf{pq}, O(\mathbf{pt} + t(n) \cdot \mathbf{pq})) - \alpha &= \Pr[0^n \notin L(R_u)] - \beta(\mathbf{pq}, O(\mathbf{pt} + t(n) \cdot \mathbf{pq})) - \alpha \\ &\quad \text{(by Claim 4.6, proven below)} \\ &\leq \Pr\left[Q_V(u_{(0)}, (b_i^{(0)})_{i \in [k]}, \rho, \mathbb{w}_0) = 1 \wedge 0^n \notin L(R_u)\right] \\ &\quad \text{(by Claim 4.7, proven below)} \\ &\leq \varepsilon_{\text{WH}}(O(\mathbf{vq} \cdot t(n)), O(\mathbf{vt} + \mathbf{vq} \cdot t(n) + \mathbf{vq} \cdot t(n))) \quad . \\ &\quad \text{(by Claim 4.8, proven below)} \end{aligned}$$

We finish our proof by proving the missing claims.

Claim 4.6.

$$\Pr[0^n \notin L(R_u)] = \left(1 - \frac{1}{2^{t(n)}}\right)^{2^{t(n)}}.$$

Proof. We have

$$\begin{aligned} \Pr[0^n \notin L(R_u)] &= \Pr\left[\forall \mathbb{w} \in \{0, 1\}^{t(n)}, (0^n, \mathbb{w}) \notin R_u\right] \quad \text{(by Definition 3.4)} \\ &= \prod_{\mathbb{w} \in \{0, 1\}^{t(n)}} \Pr[(0^n, \mathbb{w}) \notin R_u] \\ &\quad \text{(by Definition 4.2, the events } \{(0^n, \mathbb{w}) \in R_u\}_{\mathbb{w} \in \{0, 1\}^{t(n)}} \text{ are mutually independent)} \\ &= \prod_{\mathbb{w} \in \{0, 1\}^{t(n)}} \left(1 - \frac{1}{2^{t(n)}}\right) = \left(1 - \frac{1}{2^{t(n)}}\right)^{2^{t(n)}}. \\ &\quad ((0^n, \mathbb{w}) \in R_u \text{ iff for every } i \in [t(n)], u(\mathbb{w} \| e_{t(n), i})_1 = 0) \end{aligned}$$

□

Claim 4.7.

$$\Pr\left[Q_V(u_{(0)}, (b_i^{(0)})_{i \in [k]}, \rho, \mathbb{w}_0) = 1 \wedge 0^n \notin L(R_u)\right] \geq \Pr[0^n \notin L(R_u)] - \beta(\mathbf{pq}, O(\mathbf{pt} + t(n) \cdot \mathbf{pq})) - \alpha.$$

Proof. We define a malicious prover $\tilde{\mathcal{P}}$ as follows.

$\tilde{\mathcal{P}}^u(\mathbb{x})$:

1. Sample $\zeta \leftarrow \{0, 1\}^{\text{pr}}$.
2. Sample $\mathbb{w}_0 \leftarrow \{0, 1\}^{t(n)}$.
3. For $i \in [k]$:
 - (a) Simulate $b_i := \mathcal{P}^{\text{Insert}(u, \mathbb{w}_0)}(1^\lambda, (\mathbb{x}, \mathbb{w}_0), (a_j)_{j \in [i-1]}; \zeta)$ by answering each oracle query q of $\mathcal{P}$ as follows.
 - i. Compute $\mathbf{a} := \text{Insert}(u, \mathbb{w}_0)(q)$ as follows.
 - A. Query the oracle at q : $\mathbf{a} := u(q)$.
 - B. Parse q as $(\mathbb{w}', e_{t(n), i})$.
 - C. If $\mathbb{w}' = \mathbb{w}_0$ and $i \in [t(n)]$, set $\mathbf{a}_1 := 0$.
 - ii. Answer $\mathcal{P}$'s oracle call with $\mathbf{a}$.
 - (b) Output b_i .
 - (c) Receive the i -th verifier message a_i .

Answering an oracle query of $\tilde{\mathcal{P}}$ takes time $O(t(n))$ and 1 query to the oracle. Therefore, $\tilde{\mathcal{P}}$ has the running time $O(\text{pt} + \text{pq} \cdot t(n))$ and makes at most pq queries to the oracle. Note that $\mathcal{P}^{u(0)}(1^\lambda, (0^n, \mathbb{w}_0))$ in EXP is equivalent to $\tilde{\mathcal{P}}^u(0^n)$.

We have

$$\begin{aligned}
 & \Pr[0^n \notin L(R_u)] - \beta(\lambda, n, \text{pq}, O(\text{pt} + \text{pq} \cdot t(n))) \\
 & \leq \Pr[0^n \notin L(R_u)] - \Pr[\mathbf{d} = 1 \wedge 0^n \notin L(R_u)] && \text{(by the soundness of ARG)} \\
 & = \Pr[\mathbf{d} = 0 \wedge 0^n \notin L(R_u)] && \text{(by the law of total probability)} \\
 & = \Pr \left[\begin{array}{l} \mathbf{d} = 0 \wedge 0^n \notin L(R_u) \\ \wedge Q_V(u_{(0)}, (b_i^{(0)})_{i \in [k]}, \rho, \mathbb{w}_0) = 1 \end{array} \right] + \Pr \left[\begin{array}{l} \mathbf{d} = 0 \wedge 0^n \notin L(R_u) \\ \wedge Q_V(u_{(0)}, (b_i^{(0)})_{i \in [k]}, \rho, \mathbb{w}_0) = 0 \end{array} \right] \\
 & && \text{(by the law of total probability)} \\
 & = \Pr \left[\begin{array}{l} \mathbf{d} = 0 \wedge 0^n \notin L(R_u) \\ \wedge Q_V(u_{(0)}, (b_i^{(0)})_{i \in [k]}, \rho, \mathbb{w}_0) = 1 \end{array} \right] + \Pr \left[\begin{array}{l} \mathbf{d}^{(0)} = 0 \wedge 0^n \notin L(R_u) \\ \wedge Q_V(u_{(0)}, (b_i^{(0)})_{i \in [k]}, \rho, \mathbb{w}_0) = 0 \end{array} \right] \\
 & \quad \text{(if } Q_V(u_{(0)}, (b_i^{(0)})_{i \in [k]}, \rho, \mathbb{w}_0) = 0, \text{ then } \mathbf{d} = \mathbf{d}^{(0)} \text{ because the verifier's view is the same)} \\
 & \leq \Pr[0^n \notin L(R_u) \wedge Q_V(u_{(0)}, (b_i^{(0)})_{i \in [k]}, \rho, \mathbb{w}_0) = 1] + \Pr[\mathbf{d}^{(0)} = 0 \wedge 0^n \notin L(R_u)] \\
 & && \text{(since omitting an event can only increase the probability)}
 \end{aligned}$$

We are left to prove that

$$\Pr[\mathbf{d}^{(0)} = 0 \wedge 0^n \notin L(R_u)] \leq \alpha. \quad (6)$$

Let the completeness adversary $\mathcal{A}(\mathbb{x})$ be as in Lemma 4.5. The left-hand side of Eq. 6 is at most

$$\Pr[\mathbf{d} = 0 \wedge (0^n, \mathbb{w}_0) \in R_u] \quad (7)$$

by Lemma 4.5 with $Z = \{0, 1\}^{\text{vr}} \times \{0, 1\}^{\text{pc}}$, $\phi(u, \mathbb{w}_0, z = (\rho, (b_i)_{i \in [k]})) = 1 - \mathcal{V}^u(1^\lambda, 0^n, (b_i)_{i \in [k]}; \rho)$,

$$\mathcal{D}_{(u, \mathbb{w}_0)} = \left\{ z = (\rho, (b_i)_{i \in [k]}) \mid \mathbf{d} \xleftarrow{\zeta_0, \rho, (b_1, a_1, \dots, a_k)} \langle \mathcal{P}^{\text{Insert}(u, \mathbb{w}_0)}(1^\lambda, (0^n, \mathbb{w}_0)), \mathcal{V}^u(1^\lambda, 0^n) \rangle \right\}$$

for $(u, \mathbb{w}_0) \in \mathcal{U}_{\lambda, n} \times (\{0, 1\}^{t(n)} \cup \{\perp\})$. We conclude the proof by observing that Equation 7 is at most α by the completeness of ARG. $\square$

Claim 4.8.

$$\Pr \left[\text{QV}(u_{(0)}, (b_i^{(0)})_{i \in [k]}, \rho, \mathbb{w}_0) = 1 \wedge 0^n \notin L(R_u) \right] \leq \varepsilon_{\text{WH}}(O(\text{vq} \cdot t(n)), O(\text{vt} + \text{vq} \cdot t(n))) . \quad (8)$$

Proof. We use the witness-hiding property of ARG to prove the claim. We define witness-hiding adversaries $\mathcal{A}_{\text{WH},1}$ and $\mathcal{A}_{\text{WH},2}$ as follows. Let M be the ND-decider for $R_{\mathcal{U}}$ that runs in time $O(t(n))$.

- $\mathcal{A}_{\text{WH},1}^u(\mathbb{x})$:
 1. Compute $\text{Witness}^{R_{\mathcal{U}}}(u, 0^n)$ by querying u as needed.
 2. If $\text{Witness}^{R_{\mathcal{U}}}(u, 0^n) = \{\mathbb{w}_0\}$ for some $\mathbb{w}_0$, output $\mathbb{w}_0$.
 3. Otherwise, output $\perp$.
- $\mathcal{A}_{\text{WH},2}^u(\mathbb{x}, \rho, \text{tr})$:
 1. Parse tr as $(b_1, a_1, \dots, b_k, a_k)$
 2. Set $S := \emptyset$.
 3. For $i \in \{1, \dots, k\}$, simulate $a_i := \mathcal{V}^u(1^\lambda, 0^n, (b_j)_{j \in [i]}; \rho)$ by answering each oracle query q of $\mathcal{V}$ as follows.
 - (a) Query u at q ; let a be the answer.
 - (b) Answer $\mathcal{V}$'s oracle call with a .
 - (c) Parse q as $(\mathbb{w}, e_{t(n),i})$ and set $S := S \cup \{\mathbb{w}\}$.
 4. For $\mathbb{w} \in S$:
 - (a) Simulate $\text{out} := M^u(\mathbb{x}, \mathbb{w})$ by querying the oracle at q and answering M 's oracle query with $u(q)$ whenever M queries the oracle at some q .
 - (b) If $\text{out} = 1$, output $\mathbb{w}$.
 5. Output $\perp$.

We now compute the time and query complexity of $\mathcal{A}_{\text{WH},2}$. Answering an oracle query of $\mathcal{V}$ requires 1 query to the oracle and $O(t(n))$ time. Running the ND-decider once requires $O(t(n))$ queries to the oracle and $O(t(n))$ time. Since the verifier makes at most vq queries to the oracle, $\mathcal{A}_{\text{WH},2}$ makes at most $O(\text{vq} \cdot t(n))$ oracle queries and runs in time $O(\text{vt} + \text{vq} \cdot t(n))$. Therefore,

$$\Pr \left[\begin{array}{l} (0^n, \mathbb{w}') \in R_u \wedge (0^n, \mathbb{w}_0) \in R_u \\ \begin{array}{l} u \leftarrow \mathcal{U}_{\lambda,n} \\ \mathbb{w}_0 \leftarrow \mathcal{A}_{\text{WH},1}^u(0^n) \\ \mathbf{d} \xleftarrow{\zeta, \rho, \text{tr}} \langle \mathcal{P}^u(1^\lambda, (0^n, \mathbb{w}_0)), \mathcal{V}^u(1^\lambda, 0^n) \rangle \\ \mathbb{w}' \leftarrow \mathcal{A}_{\text{WH},2}^u(0^n, \rho, \text{tr}) \end{array} \end{array} \right] \quad (9)$$

is at most $\varepsilon_{\text{WH}}(\lambda, n, O(\text{vq} \cdot t(n)), O(\text{vt} + \text{vq} \cdot t(n)))$. We are left to show that Equation 9 upper bounds

$$\Pr_{\text{EXP}} \left[\text{QV}(u_{(0)}, (b_i^{(0)})_{i \in [k]}, \rho, \mathbb{w}_0) = 1 \wedge 0^n \notin L(R_u) \right] . \quad (10)$$

Equation 10 equals

$$\left(\frac{2^{t(n)} - 1}{2^{t(n)}} \right) \cdot \Pr \left[\begin{array}{l} \text{QV}(u, (b_i)_{i \in [k]}, \rho, \mathbb{w}_0) = 1 \\ \wedge (0^n, \mathbb{w}_0) \in R_u \\ \begin{array}{l} u \leftarrow \mathcal{U}_{\lambda,n} \\ \mathbb{w}_0 \leftarrow \mathcal{A}_{\text{WH},1}^u(0^n) \\ \mathbf{d} \xleftarrow{\zeta, \rho, \text{tr}} \langle \mathcal{P}^u(1^\lambda, (0^n, \mathbb{w}_0)), \mathcal{V}^u(1^\lambda, 0^n) \rangle \\ \mathbb{w}' \leftarrow \mathcal{A}_{\text{WH},2}^u(0^n, \rho, \text{tr}) \\ (b_1, a_1, \dots, b_k, a_k) := \text{tr} \end{array} \end{array} \right] , \quad (11)$$

by Lemma 4.5 with $Z = \{0, 1\}^{\text{vr}} \times \{0, 1\}^{\text{pc}}$, $\phi(u, \mathbb{w}_0, z = (\rho, (b_i)_{i \in [k]})) = Q_V(u, (b_i)_{i \in [k]}, \rho, \mathbb{w}_0)$,

$$\mathcal{D}_{(u, \mathbb{w}_0)} = \left\{ z = (\rho, (b_i)_{i \in [k]}) \mid \mathbf{d} \xleftarrow{\zeta, \rho, (b_1, a_1, \dots, b_k, a_k)} \langle \mathcal{P}^u(1^\lambda, (0^n, \mathbb{w}_0)), \mathcal{V}^u(1^\lambda, 0^n) \rangle \right\}$$

for $(u, \mathbb{w}_0) \in \mathcal{U}_{\lambda, n} \times (\{0, 1\}^{t(n)} \cup \{\perp\})$. Because $\mathcal{A}_{\text{WH}, 2}$ outputs a valid witness if the verifier makes a query at a location relevant for $\mathbb{w}_0$, Equation 11 is upper bounded by Equation 9. By the equality of Equation 11 and Equation 10, we conclude that Equation 10 is at most Equation 9, and, consequently, Equation 8 holds. $\square$

5 Zero-knowledge relativized arguments in the signed oracle model

We construct a zero-knowledge argument with succinct verification for every relation in $\text{NTIME}^{\mathcal{O}}[T]$ (Definition 3.11) in the signed oracle model.

We authenticate oracle answers using the signature-card approach of [CT10]. We consider an oracle model in which a trusted party authenticates every oracle answer. Informally, a trusted party holds a signing key and, on a query q , returns the oracle answer $a = f(q)$ together with a signature on the query-answer pair (q, a) .

Definition 5.1 (Signed oracle). *Let $\mathcal{O}$ be an oracle and let $\text{SIG} = (\text{Gen}, \text{Sign}, \text{Verify})$ be a digital signature scheme as in Definition 3.22. For a function $f: X \rightarrow Y$ with $\text{key} \notin X$, a verification key vk , and signatures $\sigma = (\sigma_q)_{q \in X}$, we write $\text{Signed}(f, \text{vk}, \sigma)$ for the function defined by*

$$\text{Signed}(f, \text{vk}, \sigma)(q) := \begin{cases} (f(q), \sigma_q) & \text{if } q \in X \\ \text{vk} & \text{if } q = \text{key} \end{cases},$$

and call f the underlying oracle of $\text{Signed}(f, \text{vk}, \sigma)$. The **signed oracle** associated with $\mathcal{O}$ and SIG , denoted $\hat{\mathcal{O}}$, is the collection $\{\hat{\mathcal{O}}_{\lambda, n}\}_{\lambda, n \in \mathbb{N}}$, where $\hat{\mathcal{O}}_{\lambda, n}$ is the following distribution:

$$\left\{ \text{Signed}(f, \text{vk}, \sigma) \left| \begin{array}{l} f \leftarrow \mathcal{O}_{\lambda, n} \\ (\text{sk}, \text{vk}) \leftarrow \text{Gen}(1^\lambda) \\ \forall q \in X_{\lambda, n}: \sigma_q \leftarrow \text{Sign}(\text{sk}, (q, f(q))) \\ \sigma := (\sigma_q)_{q \in X_{\lambda, n}} \end{array} \right. \right\}.$$

In practice, the signed oracle is implemented by a trusted party that holds the signing key, has access to the oracle, and lazily signs each query-answer pair it is asked about, reusing the signature for repeated queries. The party performs no other computation, but it *sees the queries*, which in our applications contain private data; it is trusted to sign only correct query-answer pairs and to keep the signing key and the queries private.

For a relativized relation $R_{\mathcal{O}}$, we define its **signed-oracle lift** $R_{\hat{\mathcal{O}}}$ by preserving the relation of the underlying oracle: for every signed-oracle instance $\hat{f}$ with underlying oracle f ,

$$R_{\hat{f}} := R_f.$$

Thus, the signatures authenticate oracle answers, but do not change which instance-witness pairs belong to the relation.

We prove that there exists a zero-knowledge argument for the signed-oracle lift of every relation in $\text{NTIME}^{\mathcal{O}}[T]$.

Theorem 5.2. *Let $\mathcal{O}$ be an oracle with $Y_{\lambda, n} \subseteq \{0, 1\}^{\leq \text{poly}(\lambda, n)}$ and membership in $X_{\lambda, n}$ decidable in time $\text{poly}(\lambda, n)$, let $T: \mathbb{N} \rightarrow \mathbb{N}$ be a time-constructible function with $T(n) \geq n$ that is computable in time $\text{poly}(n, \log T(n))$, let SIG be a digital signature scheme that is EUF-CMA secure relative to $\mathcal{O}$ (Definition 3.22), and let $\hat{\mathcal{O}}$ be the signed oracle associated with $\mathcal{O}$ and SIG (Definition 5.1). If there exists a keyed hash family that is collision resistant relative to $\hat{\mathcal{O}}$ (Definition 3.21), then for every relativized relation $R_{\mathcal{O}} \in \text{NTIME}^{\mathcal{O}}[T]$ there exists a relativized argument ARG for its signed-oracle lift $R_{\hat{\mathcal{O}}}$ with perfect completeness, soundness error $\text{negl}(\lambda)$ against polynomial-time provers for $n, T(n) \leq \text{poly}(\lambda)$, $2\lambda + 1$ rounds, communication complexity $\text{poly}(\lambda, \log T(n))$, prover time $\text{poly}(\lambda, T(n))$, verifier time $\text{poly}(\lambda, n, \log T(n))$,*

and malicious-verifier zero-knowledge error at most $\lambda \cdot 2^{-\lambda}$ with simulator time $\text{poly}(\lambda, T(n), \text{at})$ and query complexity $\text{poly}(\lambda, T(n), \text{aq})$. The honest prover makes $O(T(n))$ oracle queries, and the honest verifier queries the oracle only for the verification key.

The hypotheses are satisfied, for example, by a keyed hash family H that is collision resistant relative to the signed oracle of the signature scheme that Corollary 3.24 builds from H : collision resistance relative to $\hat{\mathcal{O}}$ implies collision resistance relative to $\mathcal{O}$, since an adversary relative to f is also an adversary relative to every $\hat{f}$ with underlying oracle f (it discards the signatures).

Our construction follows [CT10], except that we add zero knowledge. In Section 5.1 we construct a zero-knowledge argument of knowledge for the (non-relativized) universal relation that is secure relative to any oracle relative to which a collision-resistant hash family exists; in Section 5.2 we compile it into a zero-knowledge argument for relativized NTIME relations in the signed oracle model, which yields Theorem 5.2.

5.1 A zero-knowledge argument of knowledge for the universal relation

We apply Kilian's protocol [Kil92], in the vector-commitment formulation of [CDGSY24], to the HVZK PCP of Theorem 3.28 and the hiding vector commitment scheme of Theorem 3.33.

Theorem 5.3. *Let $\mathcal{O}$ be an oracle. If there exists a keyed hash family that is collision resistant relative to $\mathcal{O}$ (Definition 3.21), then there exists an interactive argument for the universal relation R_U (Definition 3.12) with the following properties, where $(M, \mathbb{x}, t)$ denotes the instance.*

- *Perfect completeness.*
- *Knowledge soundness error $3/4 + \text{negl}(\lambda)$ relative to $\mathcal{O}$ for $|M|, |\mathbb{x}|, t \leq \text{poly}(\lambda)$ and $\text{poly}(\lambda)$ -time provers, with extraction time $\text{poly}(\lambda, t, \text{at})$.*
- *Malicious-verifier zero-knowledge error $2^{-\lambda}$ relative to $\mathcal{O}$, with simulator time $\text{poly}(\lambda, t, \text{at})$ and simulator query complexity $\text{poly}(\lambda, t, \text{aq})$.*
- *Communication complexity $\text{poly}(\lambda, \log t)$, prover time $\text{poly}(\lambda, |M| + |\mathbb{x}| + t)$, and verifier time $\text{poly}(\lambda, |M| + |\mathbb{x}| + \log t)$. The honest prover and verifier make no oracle queries.*

Proof. We instantiate Kilian's protocol [Kil92], in the vector-commitment formulation of [CDGSY24, Construction 4.1], with the PCP of Theorem 3.28 and the vector commitment scheme of Theorem 3.33, whose public parameters are sampled by the verifier and sent as its first message (Section 3.3). Perfect completeness, the communication complexity, and the prover and verifier times follow from the parameters of the PCP and the succinctness of the vector commitment scheme. Knowledge soundness follows from [CDGSY24, Theorem 5.1]. Honest-verifier zero knowledge follows from [IMSX15, Section 6.1, Remark 6.10]. One can apply the guess-and-rewind simulator of [GMW91] to the honest-verifier simulator to obtain malicious-verifier zero knowledge; instantiating the vector commitment scheme with security parameter $\lambda + O(\log \lambda + \log t)$ makes the error of this simulator at most $2^{-\lambda}$. $\square$

5.2 Proof of Theorem 5.2

We show how to generically use a malicious-verifier zero-knowledge argument of knowledge for the universal relation to construct a malicious-verifier zero-knowledge relativized argument for relativized NTIME relations in the signed oracle model.

The signature relation. Let $\mathcal{O}$ be an oracle, let $\text{SIG} = (\text{Gen}, \text{Sign}, \text{Verify})$ be a digital signature scheme that is EUF-CMA secure relative to $\mathcal{O}$, and let $\hat{\mathcal{O}}$ be the signed oracle associated with $\mathcal{O}$ and SIG , as in Theorem 5.2.

Let $R_{\mathcal{O}} \in \text{NTIME}^{\mathcal{O}}[T]$ and let M be its ND-decider (Definition 3.11), which runs in time $O(T(n))$. We focus on the instances of the universal relation $R_{\mathcal{U}}$ (Definition 3.12) of the form $\mathbb{x}_{\text{sig}} := (M_{\text{sig}}, (1^\lambda, \mathbb{x}, \text{vk}), t)$, where M_{sig} is the machine that, on input $((1^\lambda, \mathbb{x}, \text{vk}), (\mathbb{w}, ((q_i, a_i, \sigma_i))_i))$, runs M on $(\mathbb{x}, \mathbb{w})$, answering queries outside $X_{\lambda, n}$ with the empty string and its i -th query in $X_{\lambda, n}$ with a_i after checking that the query equals q_i , checks that $\text{Verify}(\text{vk}, (q_i, a_i), \sigma_i) = 1$ for every i , and accepts if and only if M accepts and all checks pass, and $t := \text{poly}(\lambda, T(n))$ is a bound on the running time of M_{sig} . The machine M_{sig} does not query the oracle: it ensures that the oracle trace is signed, not that its answers agree with f .

Construction. Let $\text{ARG}_{\text{sig}} = (\mathcal{P}_{\text{sig}}, \mathcal{V}_{\text{sig}})$ be a malicious-verifier zero-knowledge argument of knowledge for $R_{\mathcal{U}}$ relative to $\widehat{\mathcal{O}}$. We construct $\text{ARG} = (\mathcal{P}, \mathcal{V})$ for $R_{\widehat{\mathcal{O}}}$ as follows. On common input $(1^\lambda, \mathbb{x})$ and prover witness $\mathbb{w}$:

1. Both $\mathcal{P}$ and $\mathcal{V}$:
 - (a) Query $\widehat{f}(\text{key})$ to obtain vk .
 - (b) Set $\mathbb{x}_{\text{sig}} := (M_{\text{sig}}, (1^\lambda, \mathbb{x}, \text{vk}), t)$.
2. $\mathcal{P}$:
 - (a) Run M on $(\mathbb{x}, \mathbb{w})$, answering its i -th query $q_i \in X_{\lambda, n}$ with a_i , where $(a_i, \sigma_i) := \widehat{f}(q_i)$, and its other queries with the empty string.
 - (b) Set $\mathbb{w}_{\text{sig}} := (\mathbb{w}, ((q_i, a_i, \sigma_i))_i)$.
3. $\mathcal{P}$ and $\mathcal{V}$ run $d \leftarrow \langle \mathcal{P}_{\text{sig}}(1^\lambda, \mathbb{x}_{\text{sig}}, \mathbb{w}_{\text{sig}}), \mathcal{V}_{\text{sig}}(1^\lambda, \mathbb{x}_{\text{sig}}) \rangle$.
4. $\mathcal{V}$ outputs d .

Analysis. We instantiate ARG_{sig} with the argument of Theorem 5.3 relative to $\widehat{\mathcal{O}}$, which exists by the hypothesis of Theorem 5.2, and prove that ARG has the following properties; Theorem 5.2 then follows by sequential repetition.

- Perfect completeness.
- Soundness error $3/4 + \text{negl}(\lambda)$ for $n, T(n) \leq \text{poly}(\lambda)$ and $\text{poly}(\lambda)$ -time provers.
- Malicious-verifier zero-knowledge error $2^{-\lambda}$, with simulator time $\text{poly}(\lambda, T(n), \text{at})$ and simulator query complexity $\text{poly}(\lambda, T(n), \text{aq})$.
- Three rounds, communication complexity $\text{poly}(\lambda, \log T(n))$, prover time $\text{poly}(\lambda, T(n))$, and verifier time $\text{poly}(\lambda, n, \log T(n))$. The honest prover makes $O(T(n))$ oracle queries, one for the verification key and at most one per oracle query of M , and the honest verifier queries the oracle once, for the verification key.

Completeness and efficiency. Perfect completeness follows from the perfect correctness of SIG and the perfect completeness of ARG_{sig} . The instance $\mathbb{x}_{\text{sig}}$ has length $O(\lambda + n + |\text{vk}| + \log t)$ and time bound $t = \text{poly}(\lambda, T(n))$, so the round complexity, the communication complexity, and the prover and verifier times of ARG_{sig} are as claimed; beyond running $\mathcal{V}_{\text{sig}}$, the verifier $\mathcal{V}$ spends $\text{poly}(\lambda, n, \log T(n))$ time. Since the honest parties of ARG_{sig} make no oracle queries, the honest prover makes $O(T(n))$ oracle queries, one for the verification key and at most one per oracle query of M , and the honest verifier queries the oracle once, for the verification key.

Soundness. Fix $\lambda, n \in \mathbb{N}$ with $n, T(n) \leq \text{poly}(\lambda)$, $f \in \mathcal{O}_{\lambda, n}$, an instance $\mathbb{x} \notin L(R_f)$, and a $\text{poly}(\lambda)$ -time malicious prover $\widehat{\mathcal{P}}$ for ARG . Let $\widehat{f} \leftarrow \widehat{\mathcal{O}}_{\lambda, n}$ be a signed oracle with underlying oracle f , and consider the experiment

$$\left\{ \begin{array}{l} d \xleftarrow{\text{tr}} \langle \widehat{\mathcal{P}}\widehat{f}(\mathbb{x}), \mathcal{V}\widehat{f}(1^\lambda, \mathbb{x}) \rangle \\ \mathbb{w}_{\text{sig}} \leftarrow \mathcal{E}\widehat{f}, \widehat{\mathcal{P}}(\mathbb{x})(1^\lambda, \mathbb{x}_{\text{sig}}, \text{tr}, d) \end{array} \right\},$$

where $\mathcal{E}$ is the extractor of ARG_{sig} . Since $\mathcal{V}$ only forwards the messages of $\mathcal{V}_{\text{sig}}$, $\tilde{\mathcal{P}}$ is a malicious prover for ARG_{sig} on $\mathbb{x}_{\text{sig}}$, so by the knowledge soundness of ARG_{sig} relative to $\hat{\mathcal{O}}$ (Theorem 5.3),

$$\Pr[d = 1 \wedge (\mathbb{x}_{\text{sig}}, \mathbb{w}_{\text{sig}}) \notin R_U] \leq 3/4 + \text{negl}(\lambda) .$$

We show that the probability that $d = 1$ and $(\mathbb{x}_{\text{sig}}, \mathbb{w}_{\text{sig}}) \in R_U$ is $\text{negl}(\lambda)$, which gives the claimed soundness error. If $d = 1$ and $(\mathbb{x}_{\text{sig}}, \mathbb{w}_{\text{sig}}) \in R_U$ with $\mathbb{w}_{\text{sig}} = (\mathbb{w}, ((q_i, \mathbf{a}_i, \sigma_i))_i)$, then $\mathbf{a}_i \neq f(q_i)$ for some i , since otherwise the run of M inside M_{sig} is the run of M^f on $(\mathbb{x}, \mathbb{w})$, which accepts, so $(\mathbb{x}, \mathbb{w}) \in R_f$; moreover $\text{Verify}(\text{vk}, (q_i, \mathbf{a}_i), \sigma_i) = 1$, while f signs only messages of the form $(q, f(q))$. Hence σ_i is a forgery, and the following EUF-CMA adversary outputs it.

- $\mathcal{A}^{f, \text{Sign}(\text{sk}, \cdot)}(1^\lambda, \text{vk})$:
 1. Set $\mathbb{x}_{\text{sig}} := (M_{\text{sig}}, (1^\lambda, \mathbb{x}, \text{vk}), t)$.
 2. Run the above experiment, answering each oracle call $\hat{f}(\text{key})$ with vk and each oracle call $\hat{f}(q)$ with $(f(q), \text{Sign}(\text{sk}, (q, f(q))))$, where the signature of each query is requested once and reused for repeated queries.
 3. If $d = 0$ or $(\mathbb{x}_{\text{sig}}, \mathbb{w}_{\text{sig}}) \notin R_U$, output $\perp$. Otherwise, output $((q_i, \mathbf{a}_i), \sigma_i)$ for the first i with $\mathbf{a}_i \neq f(q_i)$.

The answers of $\mathcal{A}$ to the oracle calls are distributed exactly as $\hat{f}$, so $\mathcal{A}$ forges exactly when $d = 1$ and $(\mathbb{x}_{\text{sig}}, \mathbb{w}_{\text{sig}}) \in R_U$. Since $\mathcal{A}$ runs in time $\text{poly}(\lambda)$ (Theorem 5.3 bounds the extraction time), this probability is $\text{negl}(\lambda)$ by the EUF-CMA security of SIG relative to $\mathcal{O}$.

Zero knowledge. The simulator for ARG queries $\hat{f}(\text{key})$, computes $\mathbb{x}_{\text{sig}}$ as the honest verifier would, and runs the simulator of ARG_{sig} on $\mathbb{x}_{\text{sig}}$ with the malicious verifier. Therefore, the zero knowledge of ARG follows from that of ARG_{sig} (Theorem 5.3), with the same error and one additional oracle query.

Sequential repetition. Let ARG' run steps 1 and 2 of ARG once and then step 3 λ times sequentially, accepting if and only if every session accepts; the verifier sends the parameters of each session with its last message of the previous session, so ARG' has $2\lambda + 1$ rounds. Completeness, communication complexity, verifier time, and the honest parties' oracle queries are as claimed in Theorem 5.2. For soundness, run $\mathcal{E}$ after each session on the malicious prover from its state at the beginning of the session. Conditioned on the previous sessions, each session is an execution of ARG_{sig} , so the probability that every session accepts and no extraction yields a witness for $\mathbb{x}_{\text{sig}}$ is at most $(3/4 + \text{negl}(\lambda))^\lambda = \text{negl}(\lambda)$, while the probability that some extraction yields a witness for $\mathbb{x}_{\text{sig}}$ although $\mathbb{x} \notin L(R_f)$ is $\text{negl}(\lambda)$, since $\mathcal{A}$ above, run on all sessions and extractions, then outputs a forgery. Hence ARG' has soundness error $\text{negl}(\lambda)$. By the sequential composition of auxiliary-input zero knowledge [GO94], whose hybrid argument adds the errors of the sessions, ARG' has malicious-verifier zero-knowledge error at most $\lambda \cdot 2^{-\lambda}$, with simulator time and query complexity λ times those of ARG . This proves Theorem 5.2.

Remark 5.4 (Efficiency). The construction of this section demonstrates feasibility, so we kept it simple. Replacing the PCP and the vector commitment scheme by a modern hash-based succinct argument makes the construction concretely efficient: for example, Flock [BRW26] proves batched boolean computations with a constant prover overhead over native execution and a verifier whose cost is linear in a single computation but sublinear in the size of the batch, and adding zero knowledge to such systems incurs only a small additional prover cost [CFW26].

A Impossibility of witness hiding for debate

We extend Theorem 4.1 to debate, the two-prover model of scalable oversight [ICA18; BCIP24]: a debate whose honest opponent is efficient collapses to a single-prover relativized argument with the same guarantees, so the lower bound applies to it.

Relativized debates. Let $\mathcal{O}$ be an oracle. A **relativized debate** in the oracle model $\mathcal{O}$ for a relativized relation $R_{\mathcal{O}}$ is a triple of interactive oracle algorithms $\text{DEBATE} = (\mathcal{P}_1, \mathcal{P}_2, \mathcal{J})$: the *prover* $\mathcal{P}_1$ receives the witness, the *opponent* $\mathcal{P}_2$ does not, and the *verifier* $\mathcal{J}$ decides. All three have access to the same oracle function $f \in \mathcal{O}_{\lambda, n}$. They interact for k rounds as in [ICA18; BCIP24]: in round i , the prover sends b_i and the opponent sends c_i , each computed by the party from its input, its randomness, and the messages of the previous rounds; after the last round, the verifier outputs its decision $d \in \{0, 1\}$, computed from its input, its randomness, and all messages. We write $d \xleftarrow{\zeta_1, \zeta_2, \rho, \text{tr}} \langle \mathcal{P}_1^f(1^\lambda, (\mathbb{x}, \mathbb{w})), \mathcal{P}_2^f(1^\lambda, \mathbb{x}), \mathcal{J}^f(1^\lambda, \mathbb{x}) \rangle$ for this experiment, where ζ_1, ζ_2, ρ are the randomness of the three parties and $\text{tr} := (b_i, c_i)_{i \in [k]}$ is the transcript, and omit the randomness and the transcript when they are irrelevant. The efficiency measures of Section 3.3 apply to each party: vq and vt bound the queries and running time of the verifier, pq_1 and pt_1 those of the honest prover, and pq_2 and pt_2 those of the honest opponent.

Remark A.1 (Debate with a witness). [BCIP24] also consider debates for relations with a witness. In their model, the prover announces the witness at the start of the debate, so the opponent and the verifier receive it. This is the natural choice when the witness is a solution that the verifier should learn, but it rules out any privacy, since the verifier’s view contains the witness. We therefore give the witness to the prover only.

Remark A.2 (Bounded adversaries). [BCIP24] require completeness against every opponent and soundness against every cheating prover, both computationally unbounded. We require them only against query- and time-bounded opponents and cheating provers, as for arguments (Definition 3.14), and the errors in Definitions A.3 and A.4 are functions of these bounds. Since the lower bound rules out debates under these weaker requirements, it rules out debates under the requirements of [BCIP24] as well.

Completeness must hold against every opponent and soundness against every cheating prover, as in [BCIP24].

Definition A.3 (Completeness). DEBATE has **completeness error** α if for every security parameter $\lambda \in \mathbb{N}$, instance size bound $n \in \mathbb{N}$, instance $\mathbb{x} \in \{0, 1\}^n$, adversary $\mathcal{A}$, and q -query t -time opponent $\tilde{\mathcal{P}}_2$,

$$\Pr \left[\begin{array}{l} d = 0 \\ \wedge (\mathbb{x}, \mathbb{w}) \in R_f \end{array} \mid \begin{array}{l} f \leftarrow \mathcal{O}_{\lambda, n} \\ \mathbb{w} \leftarrow \mathcal{A}^f(\mathbb{x}) \\ d \leftarrow \langle \mathcal{P}_1^f(1^\lambda, (\mathbb{x}, \mathbb{w})), \tilde{\mathcal{P}}_2^f(\mathbb{x}), \mathcal{J}^f(1^\lambda, \mathbb{x}) \rangle \end{array} \right] \leq \alpha(\lambda, n, q, t) .$$

Definition A.4 (Soundness). DEBATE has **soundness error** β if for every security parameter $\lambda \in \mathbb{N}$, instance size bound $n \in \mathbb{N}$, instance $\mathbb{x} \in \{0, 1\}^n$, and q -query t -time cheating prover $\tilde{\mathcal{P}}_1$,

$$\Pr \left[\begin{array}{l} d = 1 \\ \wedge \mathbb{x} \notin L(R_f) \end{array} \mid \begin{array}{l} f \leftarrow \mathcal{O}_{\lambda, n} \\ d \leftarrow \langle \tilde{\mathcal{P}}_1^f(\mathbb{x}), \mathcal{P}_2^f(1^\lambda, \mathbb{x}), \mathcal{J}^f(1^\lambda, \mathbb{x}) \rangle \end{array} \right] \leq \beta(\lambda, n, q, t) .$$

Privacy in a debate must protect the witness from the opponent and the verifier together, since the two may cooperate. We require the weakest such guarantee, in the spirit of Definition 3.15: the joint view of the honest opponent and the honest verifier must not allow recovering a witness. The impossibility below therefore applies to every stronger requirement, such as hiding against a malicious opponent and verifier.

Definition A.5 (Witness hiding). DEBATE has **witness-hiding error** ε_{WH} if for every security parameter $\lambda \in \mathbb{N}$, instance size bound $n \in \mathbb{N}$, instance $\mathbb{x} \in \{0, 1\}^n$, adversary $\mathcal{A}_{\text{WH},1}$, and aq-query at-time adversary $\mathcal{A}_{\text{WH},2}$,

$$\Pr \left[\begin{array}{l} (\mathbb{x}, \mathbb{w}') \in R_f \\ \wedge (\mathbb{x}, \mathbb{w}) \in R_f \end{array} \left| \begin{array}{l} f \leftarrow \mathcal{O}_{\lambda,n} \\ \mathbb{w} \leftarrow \mathcal{A}_{\text{WH},1}^f(\mathbb{x}) \\ \mathbf{d} \xleftarrow{\zeta_1, \zeta_2, \rho, \text{tr}} \langle \mathcal{P}_1^f(1^\lambda, (\mathbb{x}, \mathbb{w})), \mathcal{P}_2^f(1^\lambda, \mathbb{x}), \mathcal{J}^f(1^\lambda, \mathbb{x}) \rangle \\ \mathbb{w}' \leftarrow \mathcal{A}_{\text{WH},2}^f(\mathbb{x}, \zeta_2, \rho, \text{tr}) \end{array} \right. \right] \leq \varepsilon_{\text{WH}}(\lambda, n, \text{aq}, \text{at}) .$$

Definition A.6 (Relativized DEBATE in an oracle model). Let $\mathcal{O}$ be an oracle. The complexity class

$$\text{DEBATE}^{\mathcal{O}} \left[\begin{array}{ll} \text{completeness error} & \alpha = \alpha(\lambda, n, \mathbf{q}_{\tilde{\mathcal{P}}_2}, \mathbf{t}_{\tilde{\mathcal{P}}_2}) \\ \text{soundness error} & \beta = \beta(\lambda, n, \mathbf{q}_{\tilde{\mathcal{P}}_1}, \mathbf{t}_{\tilde{\mathcal{P}}_1}) \\ \text{witness-hiding error} & \varepsilon_{\text{WH}} = \varepsilon_{\text{WH}}(\lambda, n, \text{aq}, \text{at}) \\ \text{verifier query bound} & \mathbf{vq} = \mathbf{vq}(\lambda, n) \\ \text{verifier running time} & \mathbf{vt} = \mathbf{vt}(\lambda, n) \\ \text{honest prover query bound} & \mathbf{pq}_1 = \mathbf{pq}_1(\lambda, n) \\ \text{honest prover time bound} & \mathbf{pt}_1 = \mathbf{pt}_1(\lambda, n) \\ \text{honest opponent query bound} & \mathbf{pq}_2 = \mathbf{pq}_2(\lambda, n) \\ \text{honest opponent time bound} & \mathbf{pt}_2 = \mathbf{pt}_2(\lambda, n) \end{array} \right]$$

is the set of relativized relations $R_{\mathcal{O}}$ that admit a relativized debate $(\mathcal{P}_1, \mathcal{P}_2, \mathcal{J})$ with the listed errors and efficiency measures. A parameter that is omitted is unconstrained.

Folding the opponent into the verifier. Given a relativized debate, we obtain a single-prover relativized argument by letting the verifier run the honest opponent internally.

Lemma A.7. Let $\text{DEBATE} = (\mathcal{P}_1, \mathcal{P}_2, \mathcal{J})$ be a relativized debate for $R_{\mathcal{O}}$ with errors and efficiency measures as in Definition A.6. There exists a relativized argument $\text{ARG} = (\mathcal{P}, \mathcal{V})$ for $R_{\mathcal{O}}$ with the following properties.

- Completeness error $\alpha(\lambda, n, \mathbf{pq}_2, \mathbf{pt}_2)$, soundness error β , and witness-hiding error ε_{WH} .
- Verifier query bound $\mathbf{vq} + \mathbf{pq}_2$ and verifier running time $O(\mathbf{vt} + \mathbf{pt}_2)$.
- Honest prover query bound $\mathbf{pq}_1$ and honest prover time bound $\mathbf{pt}_1$.

Proof. Let $\mathcal{P} := \mathcal{P}_1$, and let $\mathcal{V}$, with randomness (ζ_2, ρ) , answer the i -th prover message b_i with $c_i := \mathcal{P}_2^f(1^\lambda, \mathbb{x}, (b_j, c_j)_{j < i}; \zeta_2)$, which does not depend on b_i , and after the last round output the decision $\mathcal{J}^f(1^\lambda, \mathbb{x}, (b_j, c_j)_{j \in [k]}; \rho)$. The queries and running time of $\mathcal{V}$ are those of $\mathcal{P}_2$ and $\mathcal{J}$ combined, and those of $\mathcal{P}$ are those of $\mathcal{P}_1$. An interaction of ARG is an execution of DEBATE with the honest opponent, with the same randomness and transcript, and a cheating prover for ARG sees, before sending b_i , exactly the messages of the previous rounds. Hence:

- the completeness experiment of ARG (Definition 3.13) is the completeness experiment of DEBATE (Definition A.3) with the opponent $\tilde{\mathcal{P}}_2 := \mathcal{P}_2$, which makes at most $\mathbf{pq}_2$ queries and runs in time $\mathbf{pt}_2$;
- the soundness experiment of ARG (Definition 3.14) against a cheating prover is the soundness experiment of DEBATE (Definition A.4) against the same cheating prover;
- in the witness-hiding experiment of ARG (Definition 3.15), the adversary receives the instance, the verifier randomness (ζ_2, ρ) , and the transcript, which is exactly the input $(\mathbb{x}, \zeta_2, \rho, \text{tr})$ of the adversary in Definition A.5, sampled from the same distribution.

The three error bounds follow. □

The lower bound for debate. Combining Lemma A.7 with Theorem 4.1 gives the following.

Corollary A.8. *Let $t: \mathbb{N} \rightarrow \mathbb{N}$ be a time-constructible function with $t(n) \geq n$ and $\mathcal{U}$ be the random oracle over $((\{0, 1\}^{t(n) + \lceil \log t(n) \rceil + 1}, \{0, 1\}^\lambda))_{\lambda, n \in \mathbb{N}}$.*

$$\text{NTIME}^{\mathcal{U}}[t] \not\subseteq \text{DEBATE}^{\mathcal{U}} \left[\begin{array}{ll} \text{completeness error} & \alpha = \begin{cases} 1/10 & \text{if } q_{\tilde{p}_2} = \text{poly}(\lambda, t(n)) \text{ ,} \\ & t_{\tilde{p}_2} = \text{poly}(\lambda, t(n)) \\ 1 & \text{otherwise} \end{cases} \\ \text{soundness error} & \beta = \begin{cases} 1/10 & \text{if } q_{\tilde{p}_1} = \text{poly}(\lambda, t(n)) \text{ ,} \\ & t_{\tilde{p}_1} = \text{poly}(\lambda, t(n)) \\ 1 & \text{otherwise} \end{cases} \\ \text{witness-hiding error} & \varepsilon_{\text{WH}} = \begin{cases} 1/10 & \text{if } aq = \text{poly}(\lambda, t(n)) \text{ ,} \\ & at = \text{poly}(\lambda, t(n)) \\ 1 & \text{otherwise} \end{cases} \\ \text{verifier query bound} & vq = \text{poly}(\lambda, t(n)) \\ \text{verifier running time} & vt = \text{poly}(\lambda, t(n)) \\ \text{honest prover query bound} & pq_1 = \text{poly}(\lambda, t(n)) \\ \text{honest prover time bound} & pt_1 = \text{poly}(\lambda, t(n)) \\ \text{honest opponent query bound} & pq_2 = \text{poly}(\lambda, t(n)) \\ \text{honest opponent time bound} & pt_2 = \text{poly}(\lambda, t(n)) \end{array} \right] .$$

Proof. If a relation in $\text{NTIME}^{\mathcal{U}}[t]$ had a relativized debate with the parameters in the corollary, then Lemma A.7 would give a relativized argument for it with the parameters in Theorem 4.1, contradicting that theorem. □

Remark A.9. The proof of Lemma A.7 uses completeness only against the honest opponent, so Corollary A.8 holds even for debates whose completeness is required only against the honest opponent. Since the verifier of Lemma A.7 runs the honest opponent, the lemma requires the honest opponent to be efficient, as in doubly-efficient debate [BCIP24]; a debate whose honest opponent runs in more than polynomial time is not covered. Finally, Corollary A.8 concerns witness hiding; zero knowledge against the opponent and the verifier implies witness hiding up to the probability of finding a witness from the instance alone, which is negligible for the relation of Definition 4.2. Lemma 3.19 proves this implication for arguments, and the same proof applies to debates.

Acknowledgments

This work supported in part by the Ethereum Foundation and by the Advanced Research and Invention Agency (ARIA).

AI use statement

We used generative AI tools to help write some parts of the paper. The introduction and the technical overview (Sections 1 and 2) were written by the authors, with AI assistance to improve the language and to find grammatical mistakes. The lower bound and its proof (Section 4) were written entirely by the authors, apart from typo and spelling checks. For the extension to debate (Section A) and the construction with signed oracles (Section 5), the ideas originated from the authors; the first drafts of the proofs were produced by human-guided AI generation, and the current versions were modified and checked by the authors to make sure that they match the intended arguments. Additionally, we used generative AI tools to search for and summarize relevant literature, to prepare the figures, and to obtain feedback on drafts. We have reviewed all AI-assisted work: every reference identified by an LLM was checked manually by at least one of the authors, and all AI-assisted text was revised by the authors. We take responsibility for the final content of this work, including text, claims, and artifacts produced with the aid of generative AI.

References

- [APKP24] Kasra Abbaszadeh, Christodoulos Pappas, Jonathan Katz, and Dimitrios Papadopoulos. “Zero-Knowledge Proofs of Training for Deep Neural Networks”. In: *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security*. CCS ’24. 2024, pp. 4316–4330.
- [AY25] Gal Arnon and Eylon Yogev. “Towards a White-Box Secure Fiat-Shamir Transformation”. In: *Proceedings of the 45th Annual International Cryptology Conference*. CRYPTO ’25. 2025, pp. 27–56.
- [AZWG21] Cem Anil, Guodong Zhang, Yuhuai Wu, and Roger Grosse. “Learning to Give Checkable Answers with Prover–Verifier Games”. In: *arXiv preprint arXiv:2108.12099* (2021).
- [Bar20] Beth Barnes. *Debate Update: Obfuscated Arguments Problem*. AI Alignment Forum post. 2020. URL: <https://www.alignmentforum.org/posts/PJLABqQ962hZEqhdB/debate-update-obfuscated-arguments-problem>.
- [BCCT12] Nir Bitansky, Ran Canetti, Alessandro Chiesa, and Eran Tromer. “From Extractable Collision Resistance to Succinct Non-Interactive Arguments of Knowledge, and Back Again”. In: *Proceedings of the 3rd Innovations in Theoretical Computer Science Conference*. ITCS ’12. 2012, pp. 326–349.
- [BCCT13] Nir Bitansky, Ran Canetti, Alessandro Chiesa, and Eran Tromer. “Recursive Composition and Bootstrapping for SNARKs and Proof-Carrying Data”. In: *Proceedings of the 45th ACM Symposium on the Theory of Computing*. STOC ’13. 2013, pp. 111–120.
- [BCG25] Annalisa Barbara, Alessandro Chiesa, and Ziyi Guan. “Relativized Succinct Arguments in the ROM Do Not Exist”. In: *Proceedings of the 23rd Theory of Cryptography Conference*. TCC ’25. 2025, pp. 400–416.
- [BCIP24] Jonah Brown-Cohen, Geoffrey Irving, and Georgios Piliouras. “Scalable AI Safety via Doubly-Efficient Debate”. In: *Proceedings of the 41st International Conference on Machine Learning*. Vol. 235. Proceedings of Machine Learning Research. PMLR, 2024, pp. 4585–4602.

- [BCIPCLX25] Jonah Brown-Cohen, Geoffrey Irving, Georgios Piliouras, Lijie Chen, Jiawei Li, and Zhiyang Xun. *Avoiding Obfuscation with Prover-Estimator Debate*. arXiv preprint arXiv:2506.13609. 2025.
- [BG08] Boaz Barak and Oded Goldreich. “Universal Arguments and their Applications”. In: *SIAM Journal on Computing* 38.5 (2008). Preliminary version appeared in CCC ’02., pp. 1661–1694.
- [BGHSV05] Eli Ben-Sasson, Oded Goldreich, Prahladh Harsha, Madhu Sudan, and Salil Vadhan. “Short PCPs Verifiable in Polylogarithmic Time”. In: *Proceedings of the 20th Annual IEEE Conference on Computational Complexity*. CCC ’05. 2005, pp. 120–134.
- [BLR25] Erik Brynjolfsson, Danielle Li, and Lindsey Raymond. “Generative AI at Work”. In: *The Quarterly Journal of Economics* 140.2 (2025), pp. 889–942.
- [Bow+22] Samuel R. Bowman et al. *Measuring Progress on Scalable Oversight for Large Language Models*. arXiv:2211.03540. 2022.
- [BPHI25] Marie Davidsen Buhl, Jacob Pfau, Benjamin Hilton, and Geoffrey Irving. *An Alignment Safety Case Sketch Based on Debate*. arXiv preprint arXiv:2505.03989. 2025.
- [BR93] Mihir Bellare and Phillip Rogaway. “Random Oracles are Practical: A Paradigm for Designing Efficient Protocols”. In: *Proceedings of the 1st ACM Conference on Computer and Communications Security*. CCS ’93. 1993, pp. 62–73.
- [BRW26] Benedikt Bünz, Ron D. Rothblum, and William Wang. *Flock: Fast Proving for Batch Boolean Computations*. IACR Cryptology ePrint Archive, Report 2026/1329. 2026.
- [BS14] Sumeet Bajaj and Radu Sion. “TrustedDB: A Trusted Hardware-Based Database with Privacy and Data Confidentiality”. In: *IEEE Transactions on Knowledge and Data Engineering* 26.3 (2014), pp. 752–765.
- [Bur+24] Collin Burns et al. “Weak-to-Strong Generalization: Eliciting Strong Capabilities With Weak Supervision”. In: *Proceedings of the 41st International Conference on Machine Learning*. ICML ’24. 2024, pp. 4971–5012.
- [CCGOS23] Megan Chen, Alessandro Chiesa, Tom Gur, Jack O’Connor, and Nicholas Spooner. “Proof-Carrying Data from Arithmetized Random Oracles”. In: *Proceedings of the 42nd Annual International Conference on the Theory and Applications of Cryptographic Techniques*. EUROCRYPT ’23. 2023, pp. 379–404.
- [CCS22] Megan Chen, Alessandro Chiesa, and Nicholas Spooner. “On Succinct Non-interactive Arguments in Relativized Worlds”. In: *Proceedings of the 41st Annual International Conference on the Theory and Applications of Cryptographic Techniques*. EUROCRYPT ’22. 2022, pp. 336–366.
- [CD16] Victor Costan and Srinivas Devadas. *Intel SGX Explained*. IACR Cryptology ePrint Archive, Report 2016/086. 2016.
- [CDGSY24] Alessandro Chiesa, Marcel Dall’Agnol, Ziyi Guan, Nicholas Spooner, and Eylon Yogev. “Untangling the Security of Kilian’s Protocol: Upper and Lower Bounds”. In: *Proceedings of the 22nd Theory of Cryptography Conference*. TCC ’24. 2024.
- [CF24] Alessandro Chiesa and Giacomo Fenzi. “zkSNARKs in the ROM with Unconditional UC-Security”. In: *Proceedings of the 22nd Theory of Cryptography Conference*. TCC ’24. 2024, pp. 67–89.
- [CFW26] Alessandro Chiesa, Giacomo Fenzi, and Guy Weissenberg. *Zero-Knowledge IOPPs for Constrained Interleaved Codes*. IACR Cryptology ePrint Archive, Report 2026/391. 2026.
- [CGSY24] Alessandro Chiesa, Ziyi Guan, Shahar Samocha, and Eylon Yogev. “Security Bounds for Proof-Carrying Data from Straightline Extractors”. In: *Proceedings of the 22nd Theory of Cryptography Conference*. TCC ’24. 2024, pp. 464–496.
- [CKX26] Liyan Chen, Yael Tauman Kalai, and Zoe Xi. “How to Avoid Debate: Scalable AI Safety via Doubly-Efficient Interactive Proofs”. In: *Proceedings of the 43rd International Conference on Machine Learning*. ICML ’26. Full version: arXiv:2607.03561. 2026.

- [CL20] Alessandro Chiesa and Siqi Liu. “On the Impossibility of Probabilistic Proofs in Relativized Worlds”. In: *Proceedings of the 11th Innovations in Theoretical Computer Science Conference*. ITCS ’20. 2020, 57:1–57:30.
- [Cla26] Thomas Claburn. *Chat at your own risk! Data brokers are selling deeply personal bot transcripts*. The Register, <https://www.theregister.com/software/2026/03/03/chatbot-data-harvesting-yields-sensitive-personal-info/4166233>. Mar. 2026.
- [CSA18] Paul Christiano, Buck Shlegeris, and Dario Amodei. *Supervising Strong Learners by Amplifying Weak Experts*. arXiv preprint arXiv:1810.08575. 2018.
- [CT10] Alessandro Chiesa and Eran Tromer. “Proof-Carrying Data and Hearsay Arguments from Signature Cards”. In: *Proceedings of the 1st Symposium on Innovations in Computer Science*. ICS ’10. 2010, pp. 310–331.
- [CWSK24] Bing-Jyue Chen, Suppakit Waiwitlikhit, Ion Stoica, and Daniel Kang. “ZKML: An Optimizing System for ML Inference in Zero-Knowledge Proofs”. In: *Proceedings of the 19th European Conference on Computer Systems*. EuroSys ’24. 2024, pp. 560–574.
- [FS90] Uriel Feige and Adi Shamir. “Witness Indistinguishable and Witness Hiding Protocols”. In: *Proceedings of the 22nd Annual ACM Symposium on Theory of Computing*. STOC ’90. 1990, pp. 416–426.
- [FSLH25] Olive Franzese, Ali Shahin Shamsabadi, Carter Luck, and Hamed Haddadi. “Secure and Confidential Certificates of Online Fairness”. In: *Advances in Neural Information Processing Systems 38*. NeurIPS ’25. 2025, pp. 40077–40107.
- [Gar+23] Sanjam Garg, Aarushi Goel, Somesh Jha, Saeed Mahloujifar, Mohammad Mahmoody, Guru-Vamsi Policharla, and Mingyuan Wang. “Experimenting with Zero-Knowledge Proofs of Training”. In: *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*. CCS ’23. 2023, pp. 1880–1894.
- [GFSC22] Wenhao Gao, Tianfan Fu, Jimeng Sun, and Connor W. Coley. “Sample Efficiency Matters: A Benchmark for Practical Molecular Optimization”. In: *Advances in Neural Information Processing Systems 35 (NeurIPS 2022), Datasets and Benchmarks Track*. 2022.
- [GKR15] Shafi Goldwasser, Yael Tauman Kalai, and Guy N. Rothblum. “Delegating Computation: Interactive Proofs for Muggles”. In: *Journal of the ACM* 62.4 (2015), 27:1–27:64.
- [GMR89] Shafi Goldwasser, Silvio Micali, and Charles Rackoff. “The knowledge complexity of interactive proof systems”. In: *SIAM Journal on Computing* 18.1 (1989). Preliminary version appeared in STOC ’85., pp. 186–208.
- [GMW91] Oded Goldreich, Silvio Micali, and Avi Wigderson. “Proofs that Yield Nothing But Their Validity or All Languages in NP Have Zero-Knowledge Proof Systems”. In: *Journal of the ACM* 38.3 (1991). Preliminary version appeared in FOCS ’86., pp. 690–728.
- [GO94] Oded Goldreich and Yair Oren. “Definitions and properties of zero-knowledge proof systems”. In: *Journal of Cryptology* 7.1 (1994), pp. 1–32.
- [HM96] Shai Halevi and Silvio Micali. “Practical and Provably-Secure Commitment Schemes from Collision-Free Hashing”. In: *Proceedings of the 16th Annual International Cryptology Conference*. CRYPTO ’96. 1996, pp. 201–215.
- [ICA18] Geoffrey Irving, Paul Christiano, and Dario Amodei. *AI Safety via Debate*. arXiv preprint arXiv:1805.00899. 2018.
- [IM25] Geoffrey Irving and Simon Marshall. *The Need to Relativise in Debate*. AI Alignment Forum post. 2025. URL: <https://www.alignmentforum.org/posts/XycoFucvxAPhcgJQa/the-need-to-relativise-in-debate-1>.

- [IMSX15] Yuval Ishai, Mohammad Mahmoody, Amit Sahai, and David Xiao. *On Zero-Knowledge PCPs: Limitations, Simplifications, and Applications*. Manuscript, <https://www.cs.virginia.edu/~mohammad/files/papers/ZKPCPs-Full.pdf>. 2015.
- [IW14] Yuval Ishai and Mor Weiss. “Probabilistically Checkable Proofs of Proximity with Zero-Knowledge”. In: *Proceedings of the 11th Theory of Cryptography Conference*. TCC ’14. 2014, pp. 121–145.
- [JABJM24] Madura K. P. Jayatunga, Margaret Ayers, Lotte Bruens, Dhruv Jayanth, and Christoph Meier. “How successful are AI-discovered drugs in clinical trials? A first analysis and emerging lessons”. In: *Drug Discovery Today* 29.6 (2024).
- [Kat07] Jonathan Katz. “Universally Composable Multi-party Computation Using Tamper-Proof Hardware”. In: *Proceedings of the 26th Annual International Conference on the Theory and Applications of Cryptographic Techniques*. EUROCRYPT ’07. 2007, pp. 115–128.
- [KCELMB24] Jan Hendrik Kirchner, Yining Chen, Harri Edwards, Jan Leike, Nat McAleese, and Yuri Burda. *Prover–Verifier Games Improve Legibility of LLM Outputs*. arXiv preprint arXiv:2407.13692. 2024.
- [Kil92] Joe Kilian. “A note on efficient zero-knowledge proofs and arguments”. In: *Proceedings of the 24th Annual ACM Symposium on Theory of Computing*. STOC ’92. 1992, pp. 723–732.
- [KK05] Jonathan Katz and Chiu-Yuen Koo. *On Constructing Universal One-Way Hash Functions from Arbitrary One-Way Functions*. Cryptology ePrint Archive, Report 2005/328. 2005.
- [Lak26] Ravie Lakshmanan. *Claude Extension Flaw Enabled Zero-Click XSS Prompt Injection via Any Website*. The Hacker News, <https://thehackernews.com/2026/03/claude-extension-flaw-enabled-zero.html>. Mar. 2026.
- [LKEMML18] Jan Leike, David Krueger, Tom Everitt, Miljan Martic, Vishal Maini, and Shane Legg. *Scalable Agent Alignment via Reward Modeling: A Research Direction*. arXiv preprint arXiv:1811.07871. 2018.
- [LXZ21] Tianyi Liu, Xiang Xie, and Yupeng Zhang. “zkCNN: Zero Knowledge Proofs for Convolutional Neural Network Predictions and Accuracy”. In: *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*. CCS ’21. 2021, pp. 2968–2985.
- [MBW25] Iain Martin and Emily Baker-White. *Elon Musk’s xAI Published Hundreds of Thousands of Grok Chatbot Conversations*. Forbes, <https://www.forbes.com/sites/iainmartin/2025/08/20/elon-musks-xai-published-hundreds-of-thousands-of-grok-chatbot-conversations/>. Aug. 2025.
- [McK+13] Frank McKeen, Ilya Alexandrovich, Alex Berenzon, Carlos V. Rozas, Hisham Shafi, Vedvyas Shanbhogue, and Uday R. Savagaonkar. “Innovative Instructions and Software Model for Isolated Execution”. In: *Proceedings of the 2nd International Workshop on Hardware and Architectural Support for Security and Privacy*. HASP ’13. 2013.
- [Mer89] Ralph C. Merkle. “A certified digital signature”. In: *Proceedings of the 9th Annual International Cryptology Conference*. CRYPTO ’89. 1989, pp. 218–238.
- [MHKMPK21] Fan Mo, Hamed Haddadi, Kleomenis Katevas, Eduard Marin, Diego Perino, and Nicolas Kourtellis. “PPFL: Privacy-Preserving Federated Learning with Trusted Execution Environments”. In: *Proceedings of the 19th Annual International Conference on Mobile Systems, Applications, and Services*. MobiSys ’21. 2021, pp. 94–108.
- [MMZ25] Mary Maller, Nicolas Mohnblatt, and Arantxa Zapico. “IVC in the Open-and-Sign Random Oracle Model”. In: *Proceedings of the 31st International Conference on the Theory and Application of Cryptology and Information Security*. ASIACRYPT ’25. 2025, pp. 333–365.
- [Moo+23] Michael Moor, Oishi Banerjee, Zahra Shakeri Hossein Abad, Harlan M. Krumholz, Jure Leskovec, Eric J. Topol, and Pranav Rajpurkar. “Foundation models for generalist medical artificial intelligence”. In: *Nature* 616 (2023), pp. 259–265.

- [Nag25] Gal Nagli. *Wiz Research Uncovers Exposed DeepSeek Database Leaking Sensitive Information, Including Chat History*. Wiz Research, <https://www.wiz.io/blog/wiz-research-uncovers-exposed-deepseek-database-leak>. Jan. 2025.
- [NY89] Moni Naor and Moti Yung. “Universal one-way hash functions and their cryptographic applications”. In: *Proceedings of the 21st Annual ACM Symposium on Theory of Computing*. STOC ’89. 1989, pp. 33–43.
- [NZ23] Shakked Noy and Whitney Zhang. “Experimental evidence on the productivity effects of generative artificial intelligence”. In: *Science* 381.6654 (2023), pp. 187–192.
- [Pad+24] Martina Padovan et al. “ChatGPT in Occupational Medicine: A Comparative Study with Human Experts”. In: *Bioengineering* 11.1 (2024), p. 57.
- [PST17] Rafael Pass, Elaine Shi, and Florian Tramèr. “Formal Abstractions for Attested Execution Secure Processors”. In: *Proceedings of the 36th Annual International Conference on the Theory and Applications of Cryptographic Techniques*. EUROCRYPT ’17. 2017, pp. 260–289.
- [PVC18] Christian Priebe, Kapil Vaswani, and Manuel Costa. “EnclaveDB: A Secure Database Using SGX”. In: *Proceedings of the 39th IEEE Symposium on Security and Privacy*. S&P ’18. 2018, pp. 264–278.
- [Rav25] Itay Ravia. *Breaking down EchoLeak, the First Zero-Click AI Vulnerability Enabling Data Exfiltration from Microsoft 365 Copilot*. <https://www.catonetworks.com/blog/breaking-down-echoleak/>. May 2025.
- [Rom90] John Rompel. “One-way functions are necessary and sufficient for secure signatures”. In: *Proceedings of the 22nd Annual ACM Symposium on Theory of Computing*. STOC ’90. 1990, pp. 387–394.
- [Row26] Antony Rowstron. *Agentic Witnessing: Pragmatic and Scalable TEE-Enabled Privacy-Preserving Auditing*. arXiv:2604.24203. 2026.
- [Sha25] Will Shanklin. *OpenAI is removing ChatGPT conversations from Google*. Engadget, <https://www.engadget.com/ai/openai-is-removing-chatgpt-conversations-from-google-194735704.html>. Aug. 2025.
- [Sin+23] Karan Singhal et al. “Large language models encode clinical knowledge”. In: *Nature* 620 (2023), pp. 172–180.
- [SLZ24] Haochen Sun, Jason Li, and Hongyang Zhang. “zkLLM: Zero Knowledge Proofs for Large Language Models”. In: *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security*. CCS ’24. 2024, pp. 4405–4419.
- [Sto+20] Jonathan M. Stokes et al. “A Deep Learning Approach to Antibiotic Discovery”. In: *Cell* 180.4 (2020), pp. 688–702.
- [TTEGTT23] Arun James Thirunavukarasu, Darren Shu Jeng Ting, Kabilan Elangovan, Laura Gutierrez, Ting Fang Tan, and Daniel Shu Wei Ting. “Large language models in medicine”. In: *Nature Medicine* 29 (2023), pp. 1930–1940.
- [Val08] Paul Valiant. “Incrementally Verifiable Computation or Proofs of Knowledge Imply Time/Space Efficiency”. In: *Proceedings of the 5th Theory of Cryptography Conference*. TCC ’08. 2008, pp. 1–18.
- [Vel+26] Francesca Vella et al. “Application of large language models as decision support tools in occupational health and safety management: a cohort study of industrial workers”. In: *Frontiers in Public Health* 14 (2026).
- [Won+24] Felix Wong et al. “Discovery of a structural class of antibiotics with explainable deep learning”. In: *Nature* 626 (2024), pp. 177–185.
- [WSSHK24] Suppakit Waiwitlikhit, Ion Stoica, Yi Sun, Tatsunori Hashimoto, and Daniel Kang. “Trustless Audits without Revealing Data or Models”. In: *Proceedings of the 41st International Conference on Machine Learning*. ICML ’24. 2024, pp. 49808–49821.

- [Xu+25] Zuojun Xu et al. “A generative AI-discovered TNIK inhibitor for idiopathic pulmonary fibrosis: a randomized phase 2a trial”. In: *Nature Medicine* 31 (2025), pp. 2602–2610.
- [Zha+19] Alex Zhavoronkov et al. “Deep learning enables rapid identification of potent DDR1 kinase inhibitors”. In: *Nature Biotechnology* 37 (2019), pp. 1038–1040.